

Exploring Misconfigured Security Across the Mobile Platform Stack

PhD Defense

Florian Draschbacher

June 16th, 2026

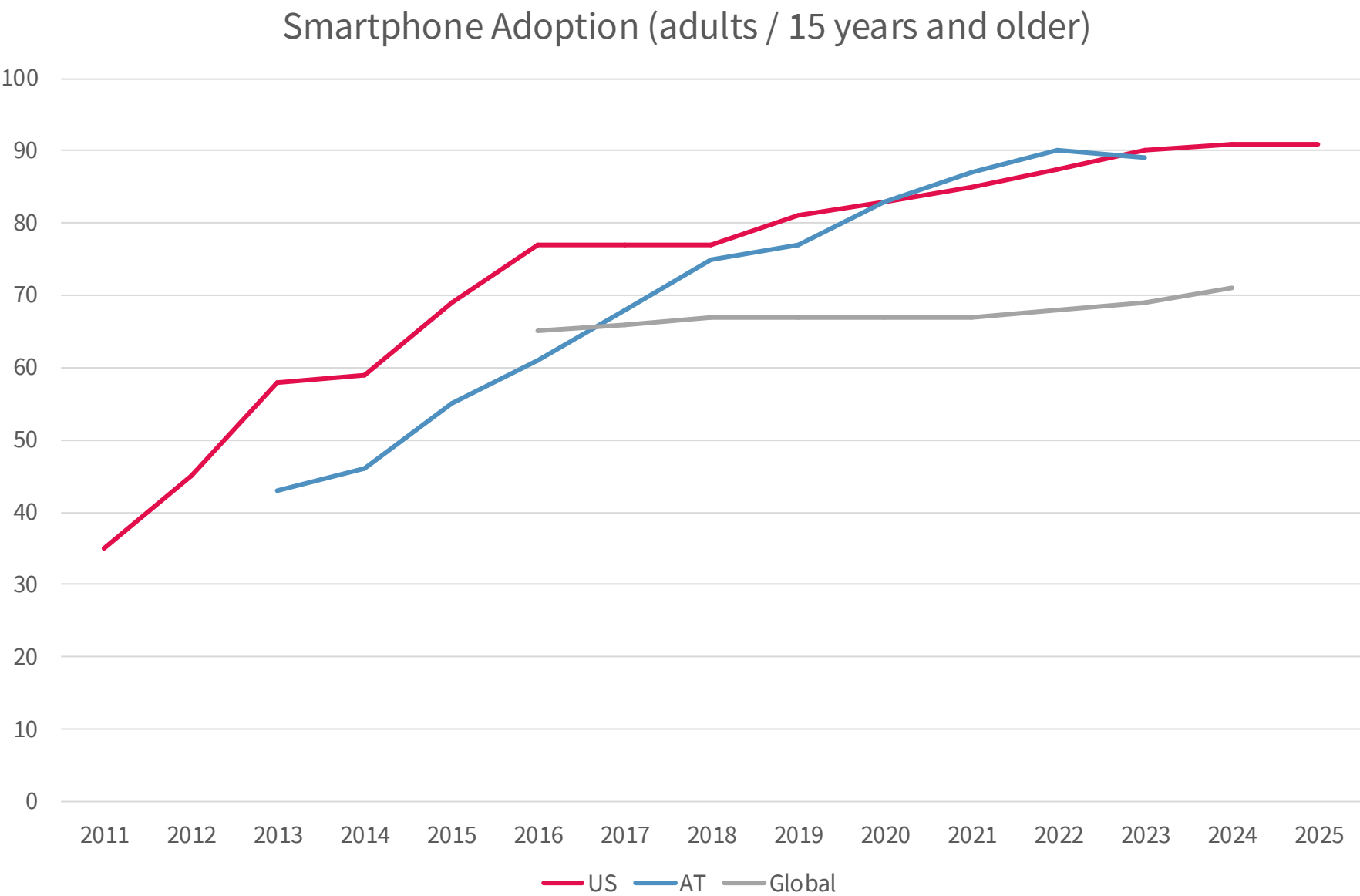
› isec.tugraz.at

Smartphones are Everywhere

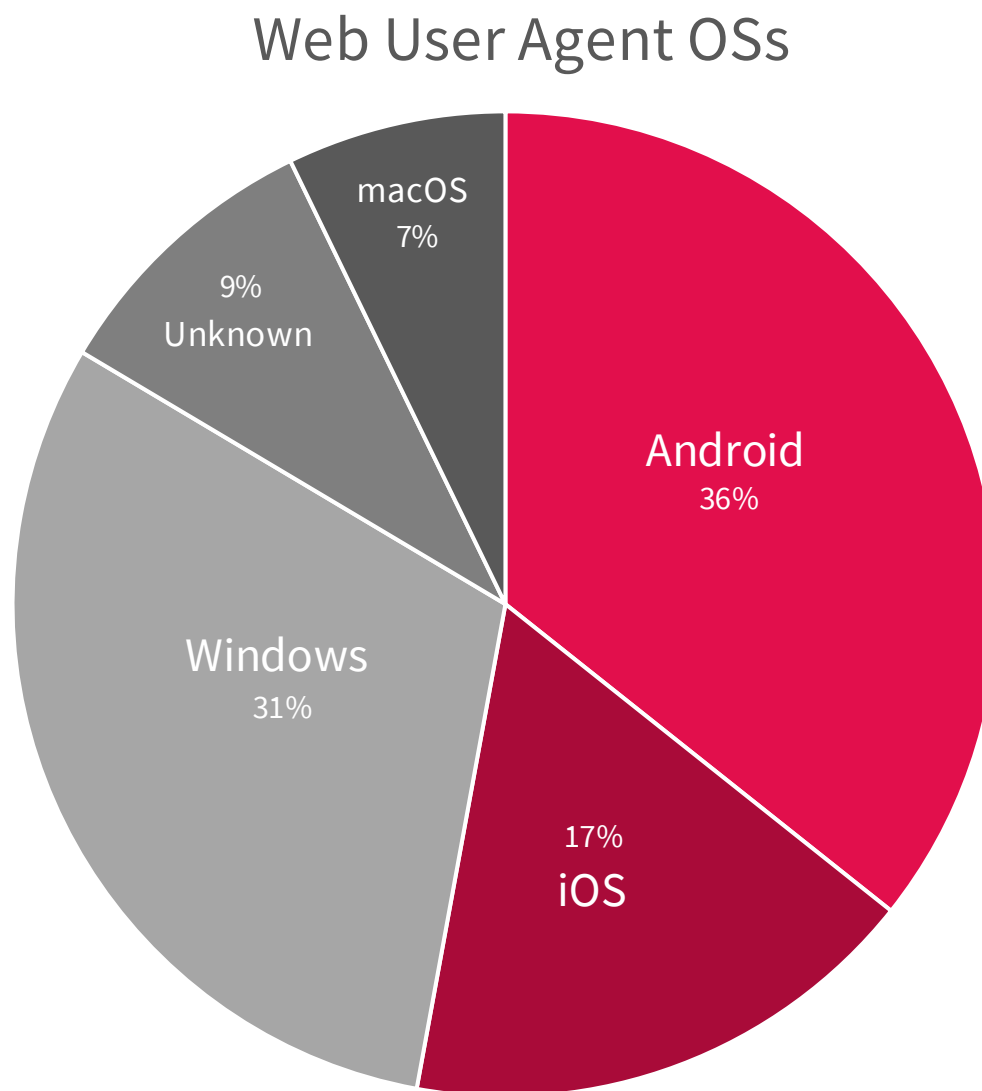
isec.tugraz.at ■



Smartphones are Everywhere

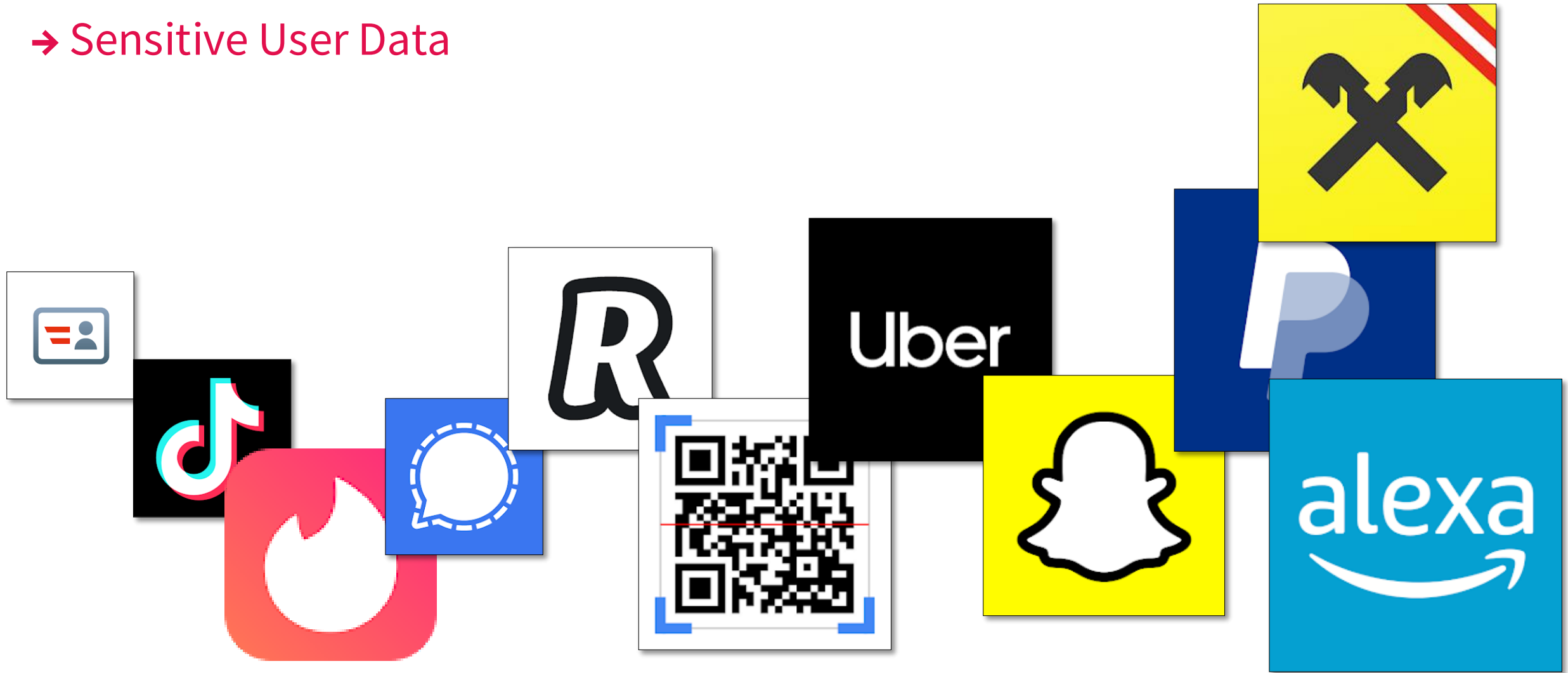


Smartphones are Everywhere



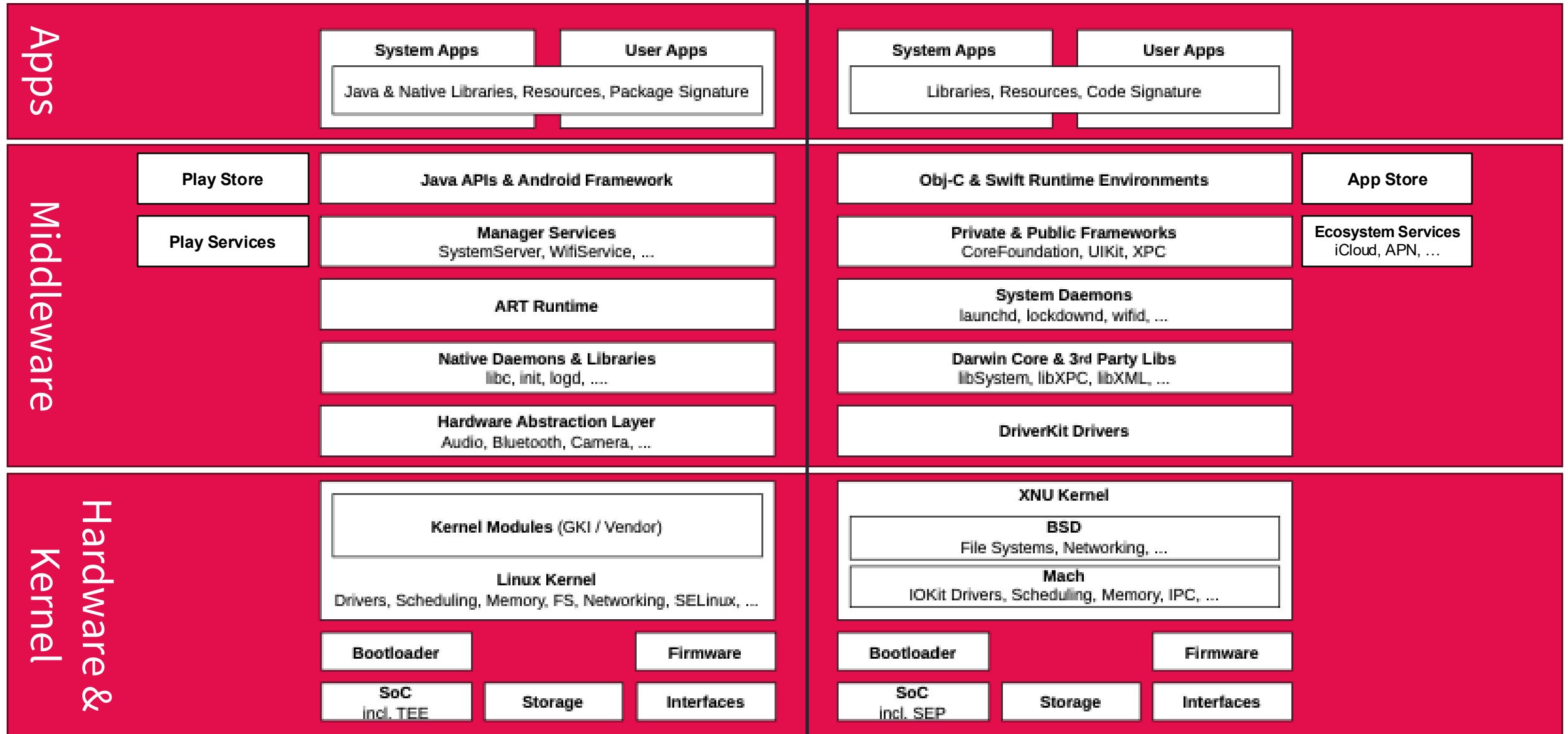
Countless Novel Applications

→ Sensitive User Data



The Mobile Platform Stack

isec.tugraz.at ■



Android 

 iOS

The Mobile Platform Stack

Apps

Middleware

Hardware & Kernel

SoC incl. TEE/SEP, Bootloader, Storage, (Connectivity) Interfaces, Firmware, Drivers, Scheduling, Memory, File Systems, Networking, SELinux, ...

isec.tugraz.at



The Mobile Platform Stack

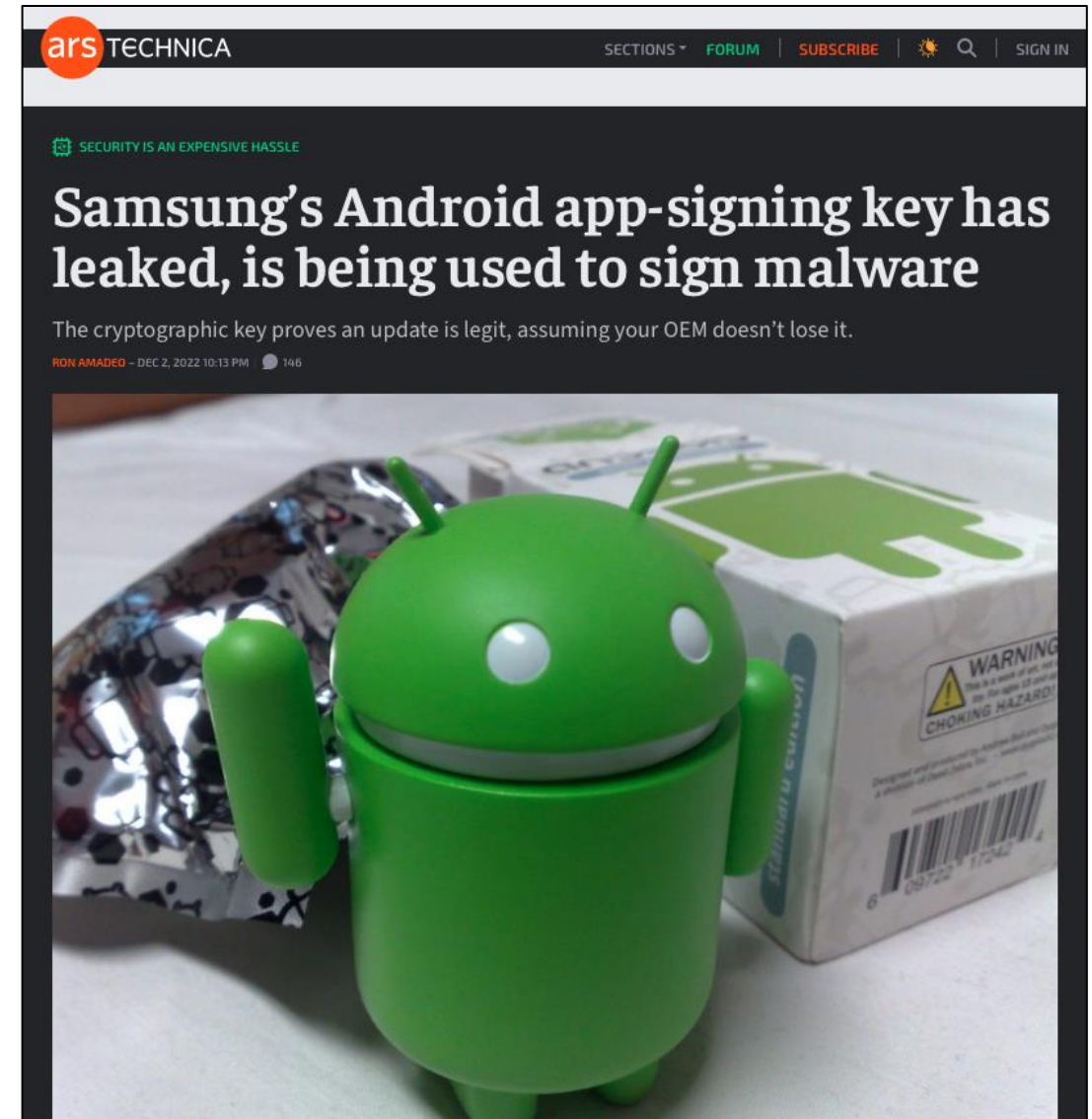
isec.tugraz.at

Apps

Middleware

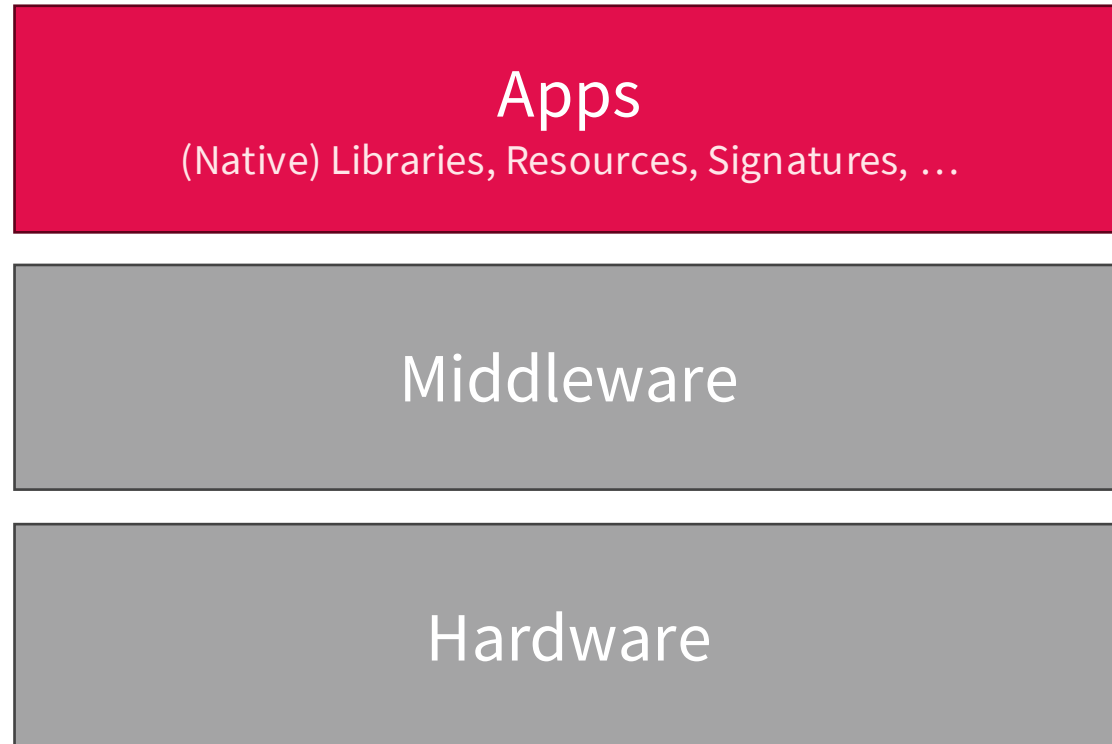
Daemons, Libraries, Runtime, Frameworks, App Stores,
Services, ...

Hardware



The Mobile Platform Stack

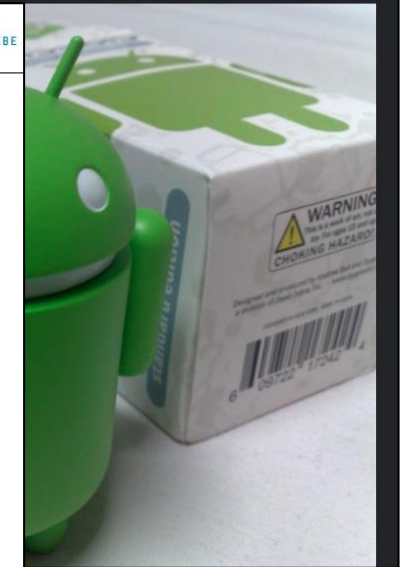
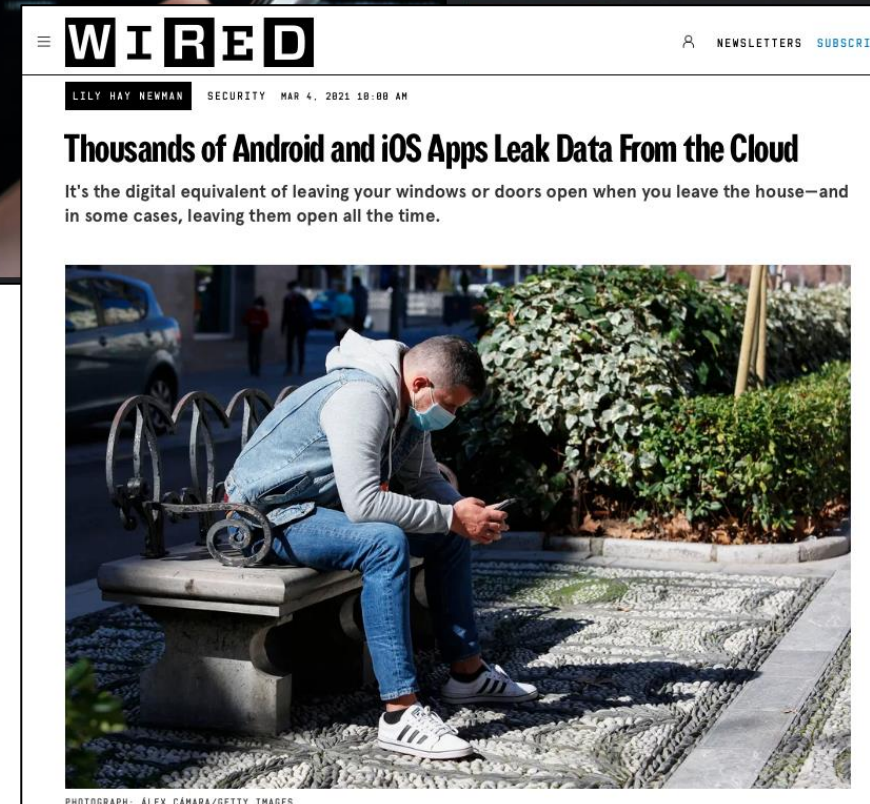
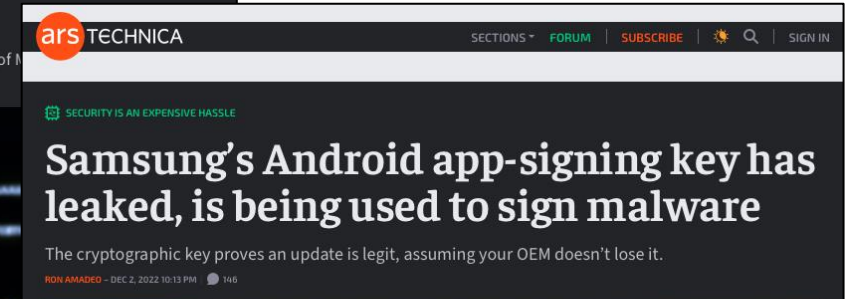
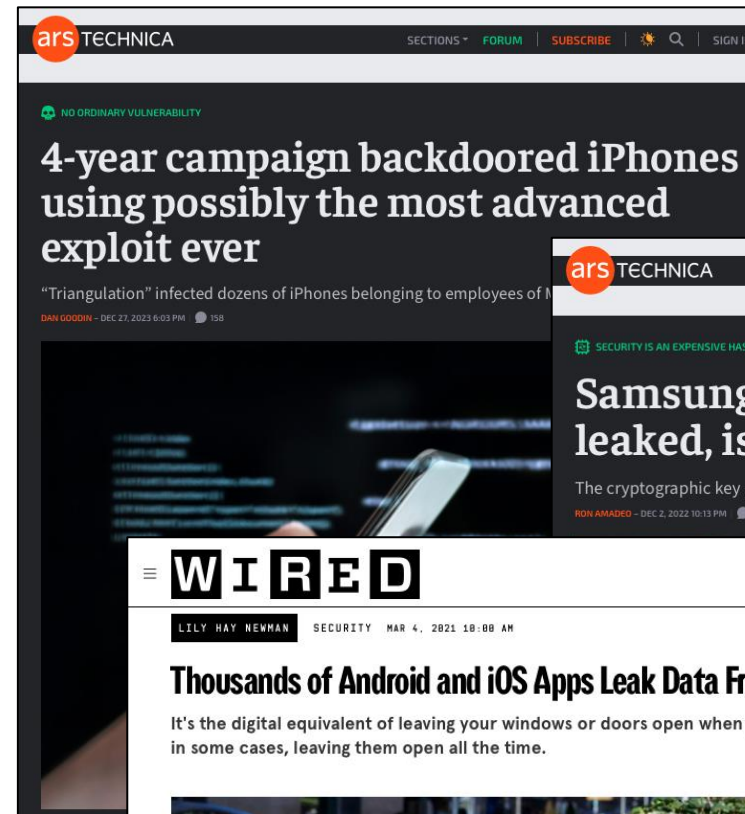
isec.tugraz.at



Misconfigured Security

isec.tugraz.at

- Security mechanisms present
 - Integrity Checks
 - Cryptography
 - Access Control
- But integration flawed
 - Incomplete
 - Ineffective
 - Bypassable



Misconfigured Security Across the Mobile Platform Stack isec.tugraz.at ■

Apps

API Misuse

❓ CryptoShield

❓ A2P2

Middleware

Supply Chains

❓ Manifest Problems

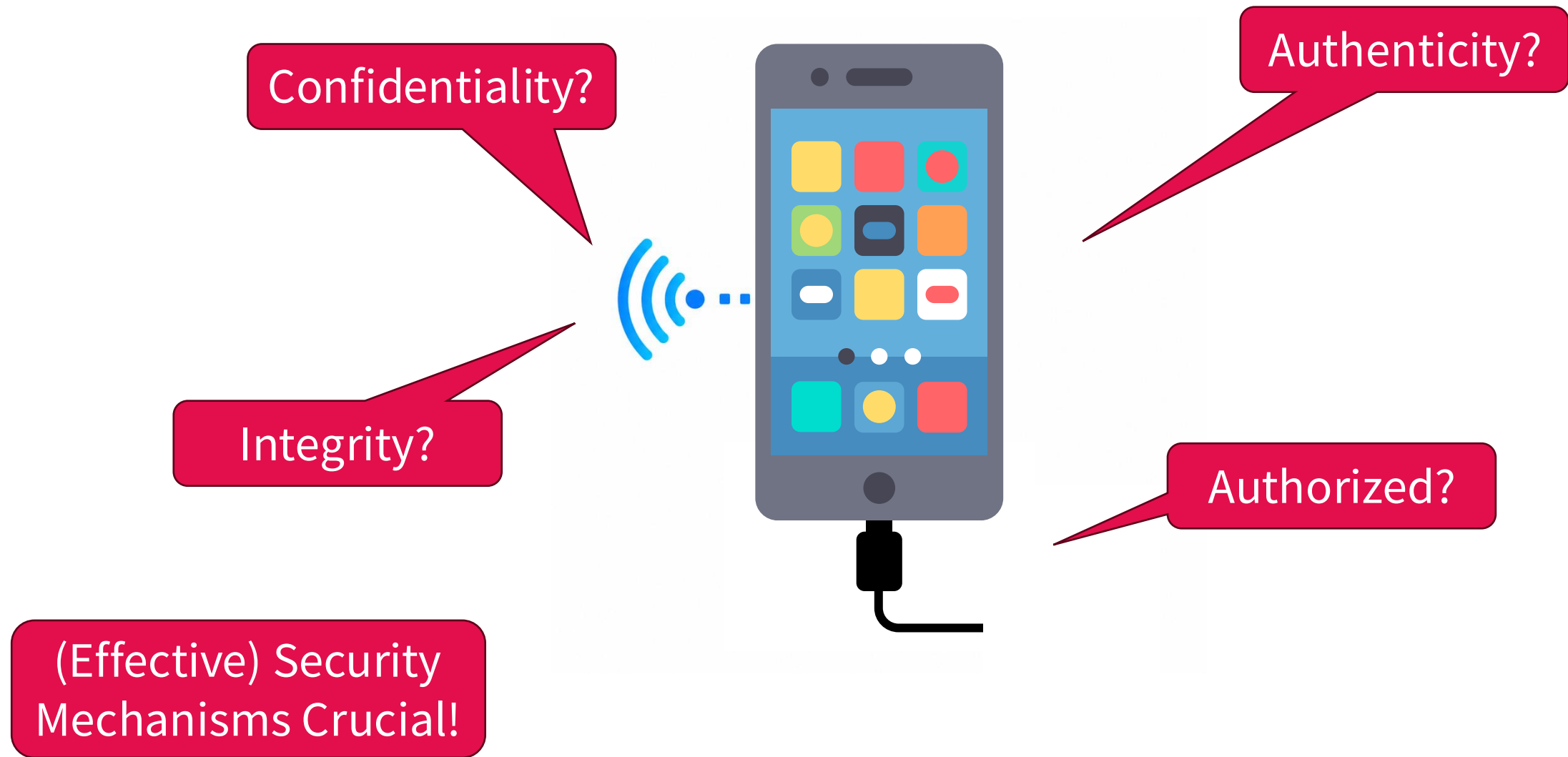
Hardware & Kernel

Communication Interfaces

❓ ChoiceJacking

❓ Clone2Pwn

Mobile Communication Interfaces



Mobile Communication Interfaces

Standardized protocols, but custom **implementations** and **services**

USB

- Charging & data
- Common attack point
- Logic-level flaws?

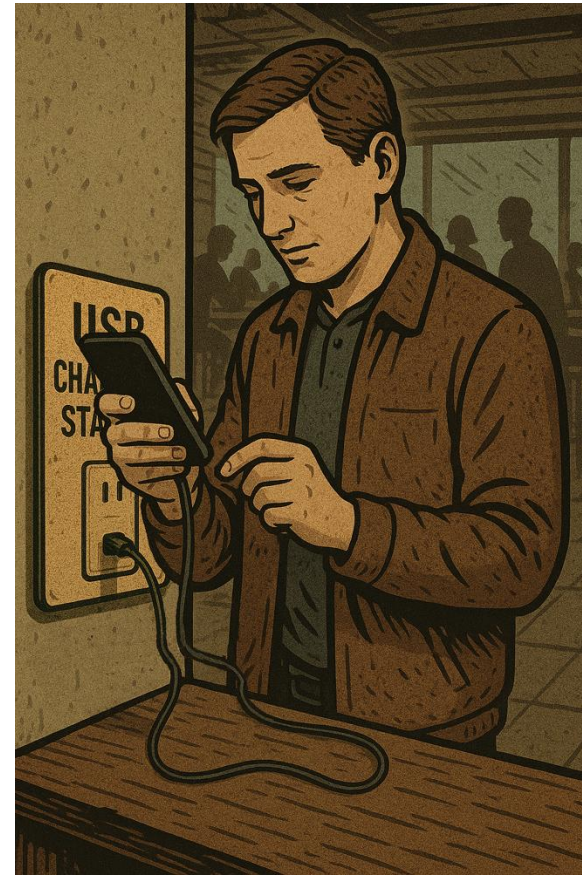
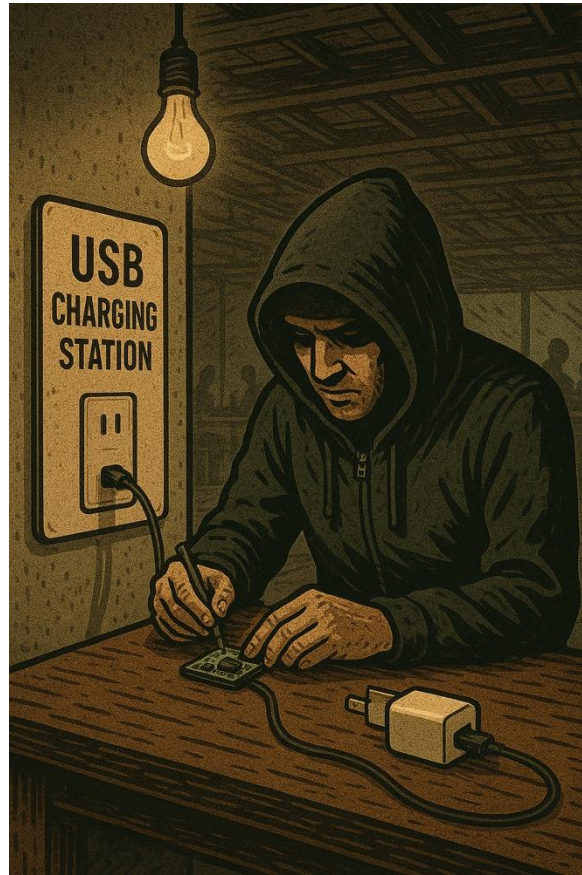
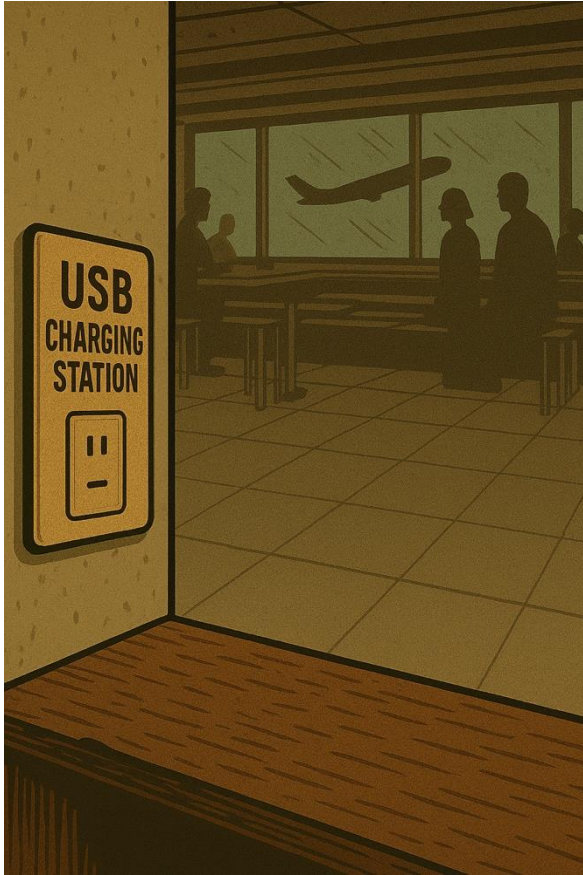
→ Authorization

Wi-Fi

- Proximity attacks
- Proprietary services
- Proper C/I/A ?

→ Data Migration Tools

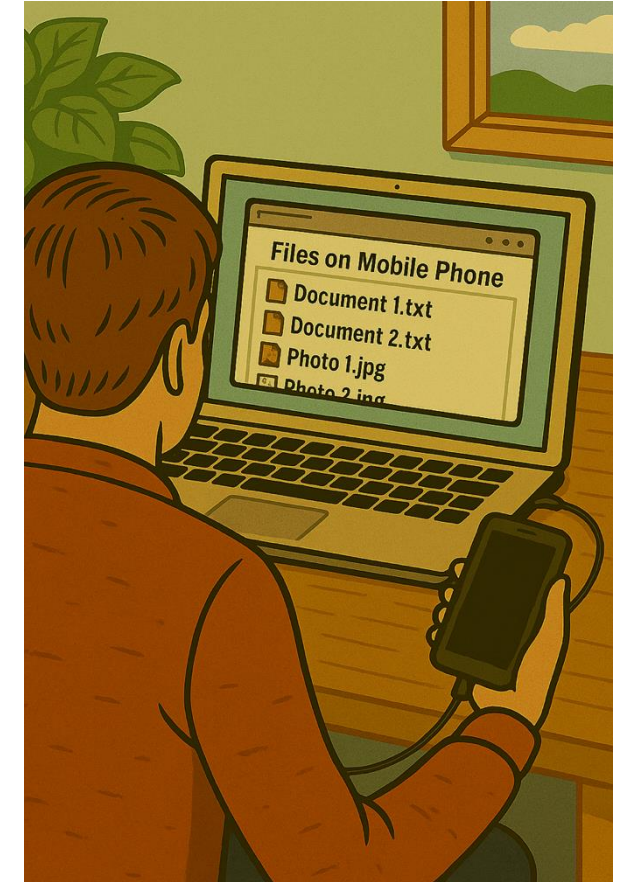
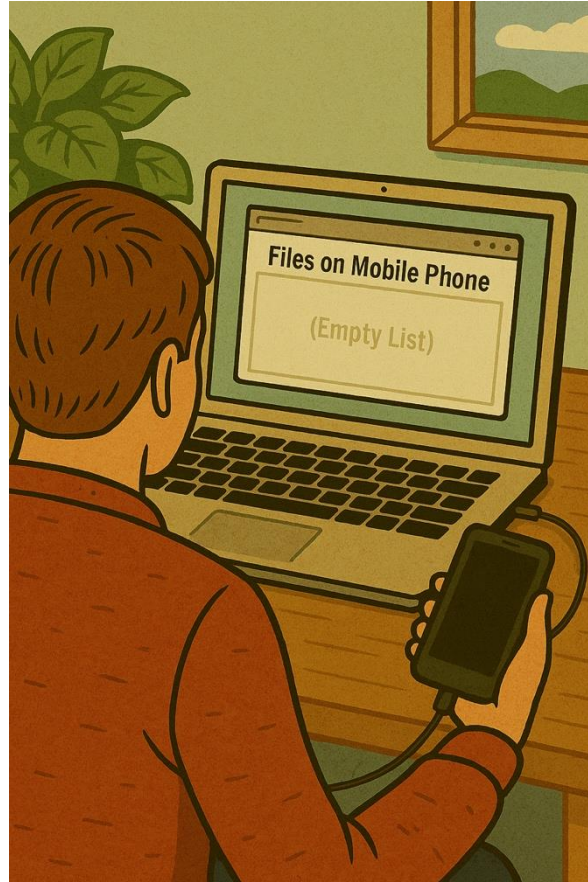
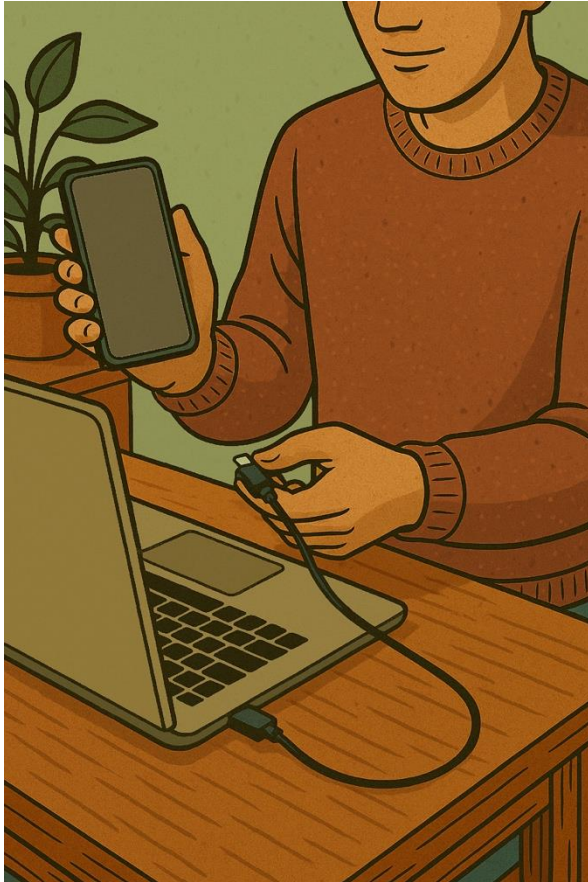
USB: JuiceJacking (~2011)



Stealthy data extraction through **manipulated phone chargers**

Airports, Train Stations, Museums, Hotel Rooms, Rental Cars, Power Banks, ...

USB: JuiceJacking Mitigations (2013-)



iOS and Android require **user consent** for USB file access

USB on Mobile

Mobile devices use multi-function USB-C ports

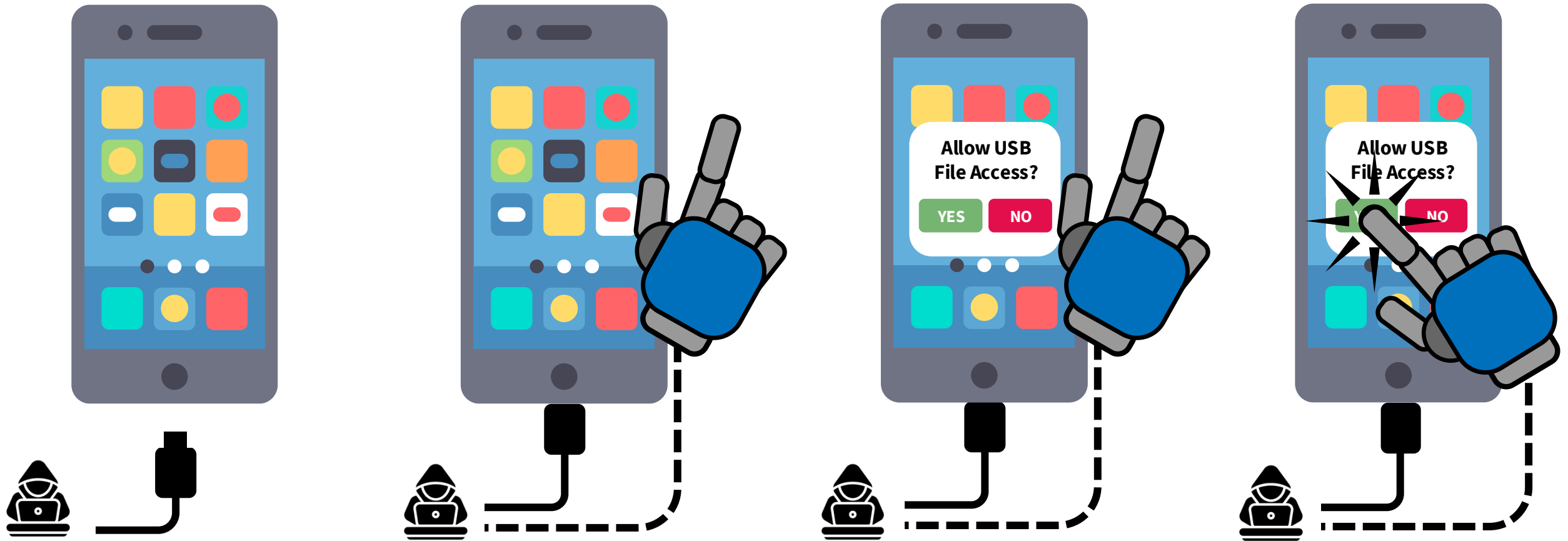
- **Power**
Charge phone or supply peripherals
- **USB Host**
Data exchange with peripherals
- **USB Device**
Data exchange with computer



Important: A USB port **either** acts as USB host or USB device at a given time

→ Foundation of JuiceJacking mitigation

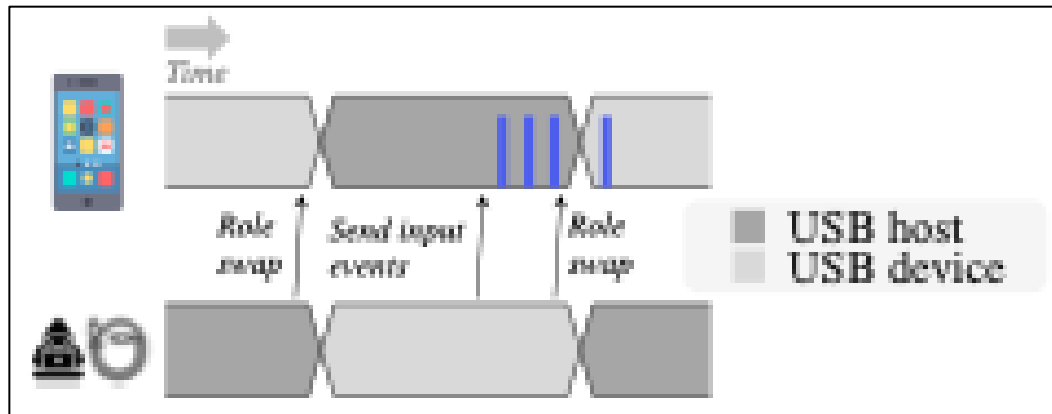
ChoiceJacking



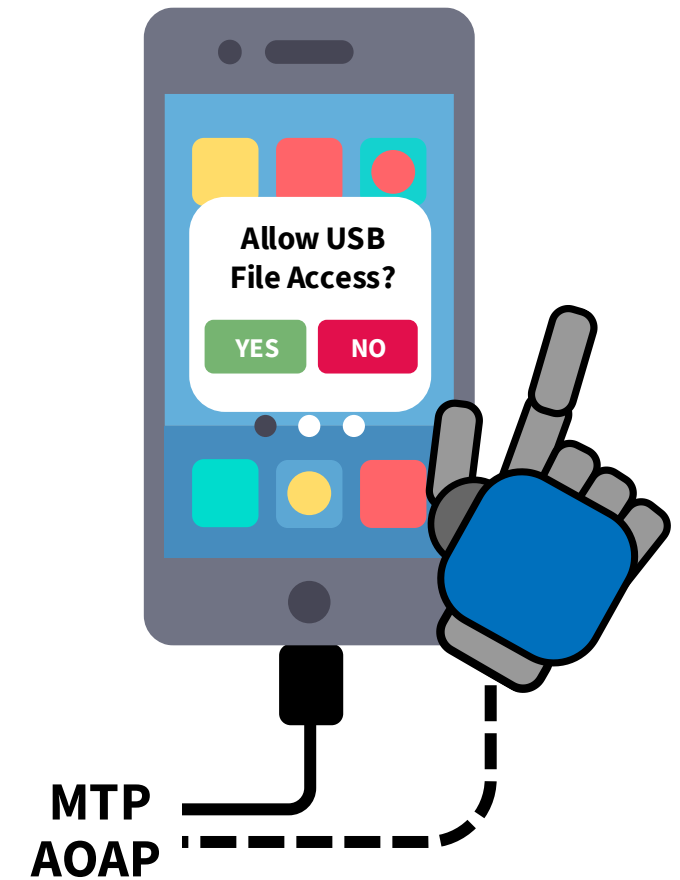
Core Principle: Establish input event channel in addition to file channel

3 Attack Techniques

- **T1:** Flaw in Android Open Accessory Protocol (AOAP)
 - Against specification: Concurrent HID events and file exchange (MTP) possible
- **T2:** Race Condition in Android input dispatcher
 - Process events after role swap

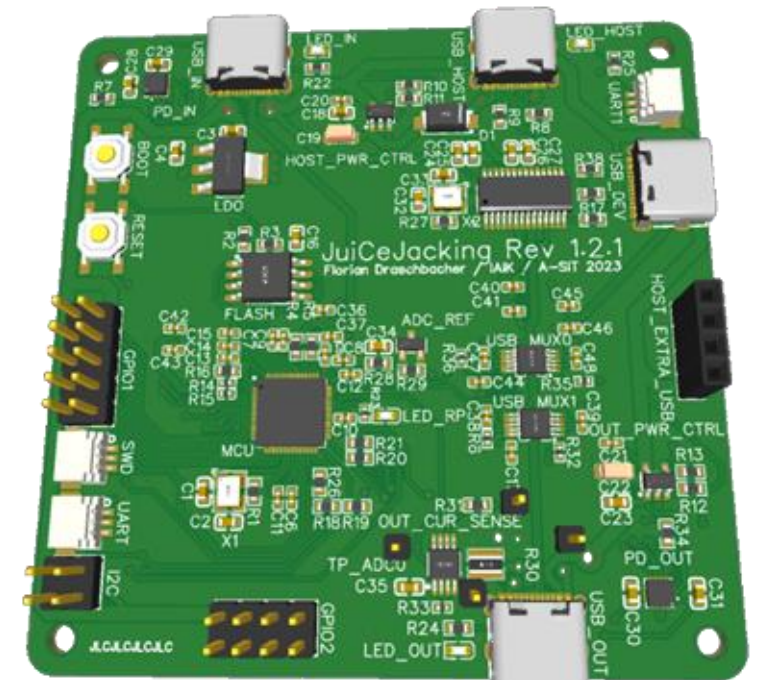


- **T3:** Pivoting via Bluetooth HID
 - iOS and Android!



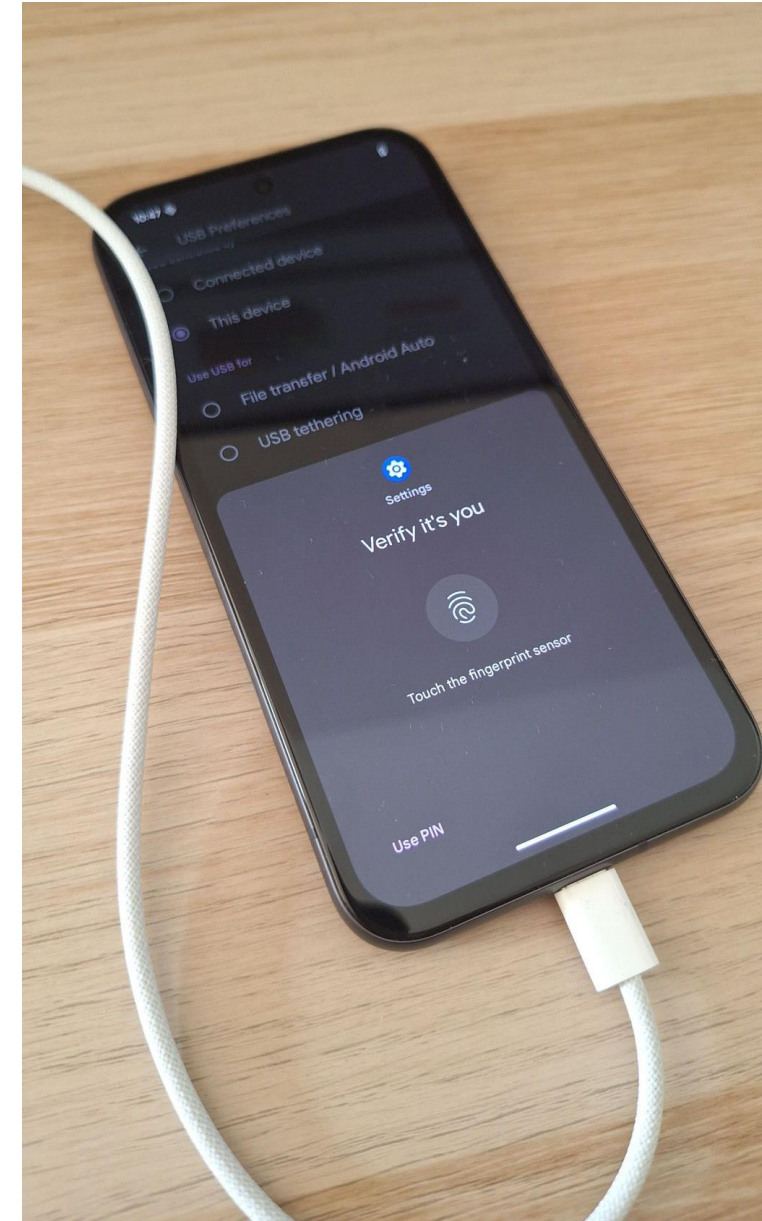
Evaluation

- 11 devices across **Android and iOS**
 - **Samsung, Xiaomi, Vivo, Oppo, Huawei, Honor, Google, Apple**
- All susceptible to at least 1 technique
- Some attacks faster than a human blink
 - **Samsung**: ~130 milliseconds
- Even worked on locked screens for some vendors



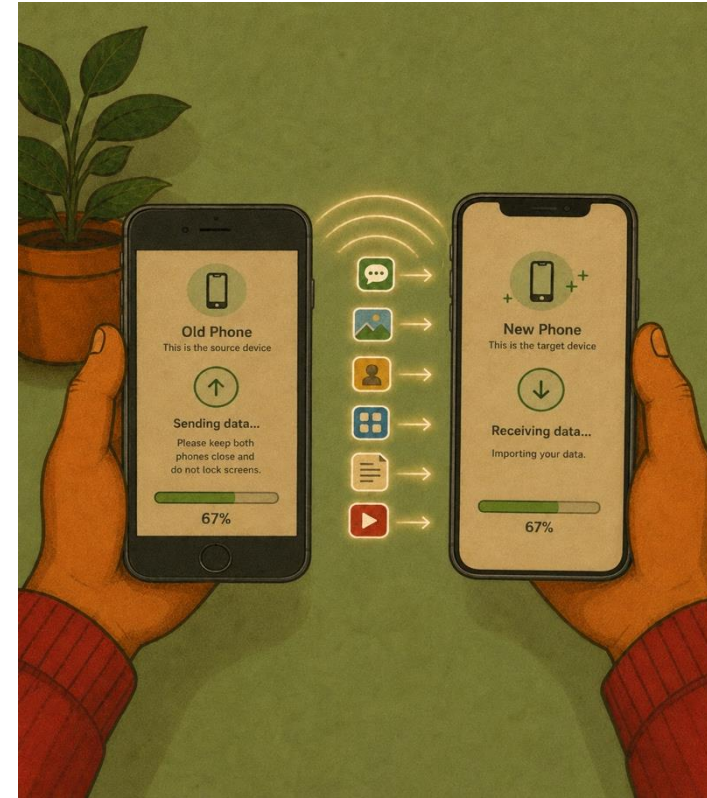
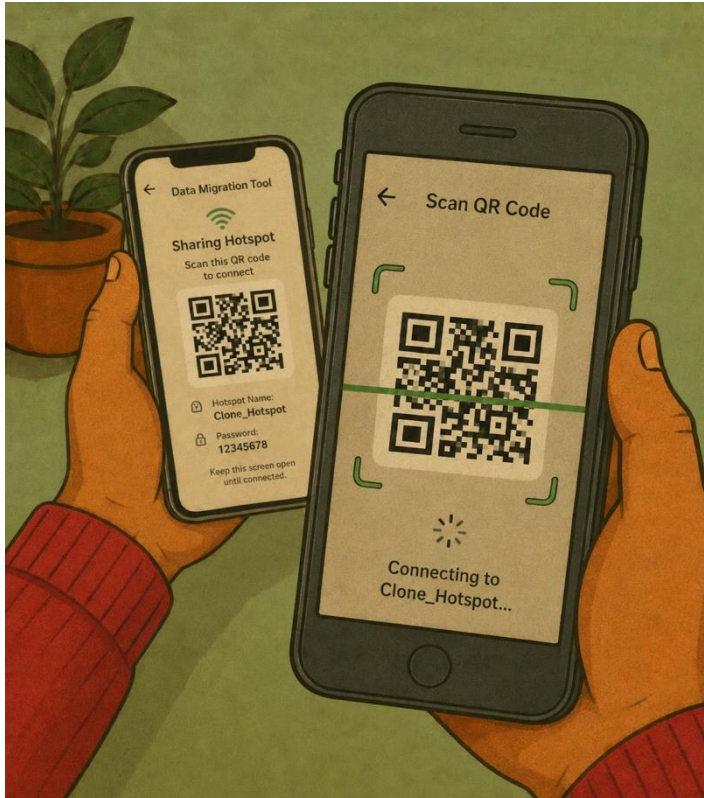
Disclosure

- Responsibly disclosed 21 security issues
 - 16 vendor-specific, 4 upstream Android, 1 iOS issue
- 5 CVEs, 1 high severity
 - Samsung, Apple, Google, ...
- Fixes in Android 15 and iOS 18.4
 - Require authentication for USB file access



Wi-Fi: Data Migration Tools

isec.tugraz.at



Android Switch

Google LLC

3.5★

1.2m reviews

10B+

Downloads



Samsung Smart Switch Mobile

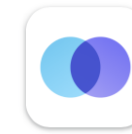
Samsung Electronics Co., Ltd.

3.9★

512k reviews

1B+

Downloads



EasyShare

PT vivo mobile Indonesia

4,6★

460k reviews

1B+

Downloads

Wi-Fi: Data Migration Tools

isec.tugraz.at ■



Vendor	App	Users	PicturesVideosDocumentsCall LogsMessagesContactsAppsApp DataWi-Fi KeysHealth Data									
			?	?	?	?	?	?	?	?	?	?
Google	Android Switch	10B+	✓	✓	-	✓	✓	✓	-	✓	✓	✓
Samsung	Smart Switch	1B+	✓	✓	✓	✓	✓	✓	✓	✓ ¹	✓ ¹	-
Vivo	EasyShare	1B+	✓	✓	✓	✓	✓	✓	✓	-	✓ ¹	-
Xiaomi	Mi Mover	100M+	✓	✓	✓	✓	✓	✓	✓	-	✓ ¹	-
Oppo	Clone Phone	10M+	✓	✓	✓	✓	✓	✓	✓	✓ ¹	✓ ¹	-
Huawei	Phone Clone	10M+	✓	✓	✓	✓	✓	✓	✓	-	-	-
Honor	Device Clone	1M+	✓	✓	✓	✓	✓	✓	✓	-	✓ ¹	-

¹ Only first-party migrations

6 Attack Scenarios (S1 - S6), 15 Checkable Requirements (R1 – R15)

Requirement	S1 (Sniff)	S2 (MITM)	S3 (Sock read)	S4 (Sock write)	S5 (FS read)	S6 (FS write)
R1: App is identifiable through Wi-Fi beacons	✓	✓	-	-	-	-
R2: Wireless protection weak	✓	-	-	-	-	-
R3: No app-level payload encryption	✓	-	✓	✓	-	-
R4: Wireless protection weak (Wi-Fi AP)	-	✓ ¹	-	-	-	-
R5: Pairing identification ambiguous (Wi-Fi Direct)	-	✓ ¹	-	-	-	-
R6: Payload encryption not derived from OOB secret	-	✓	-	-	-	-
R7: Payload integrity not protected via OOB secret	-	✓	-	-	-	-
R8: Server socket on old device accepts extra clients	-	-	✓	-	-	-
R9: No authentication for payload connections	-	-	✓	✓	-	-
R10: Wi-Fi join is device-global	-	-	-	✓	-	-
R11: Server Socket on new device accepts extra clients	-	-	-	✓	-	-
R12: No encrypted or separately transmitted checksums	-	-	-	✓	-	-
R13: User data (temporarily) stored in public file system	-	-	-	-	✓	✓
R14: No encryption for data stored in public file system	-	-	-	-	✓	✓
R15: No integrity checks for data in public file system	-	-	-	-	-	✓

¹ Either R4 or R5 is necessary for S2

Evaluation Results

App	Attack Scenario					
	Proximity (M1)		Local (M2)			
	S1	S2	S3	S4	S5	S6
Google	?	-	-	-	-	-
Samsung	-	?	-	-	?	?
Vivo	?	?	?	-	-	-
Xiaomi	?	?'	-	-	-	-
Oppo	? ¹	-	-	? ²	-	-
Huawei	?	?	-	-	-	-
Honor	?	?	-	-	-	-

¹ Most data types ² Only first-party migrations

- 8 CVEs, 5 high severity
- Fixes rolled out / promised

5 / 7 tools

Proximity attacker can get code execution

6 / 7 tools

Proximity attacker can extract data

2 / 7 tools

Local attacker can extract data

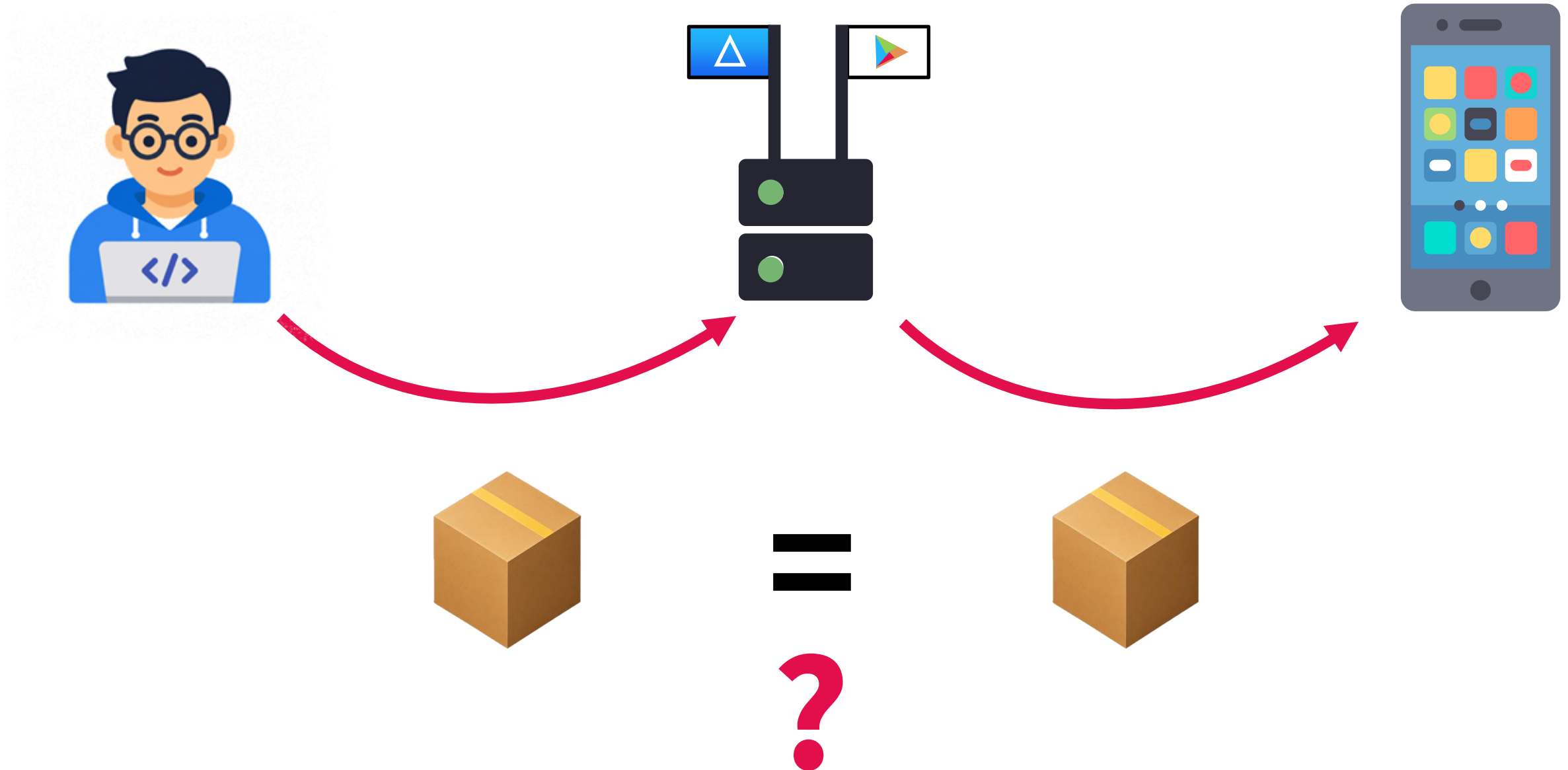
2 / 7 tools

Local attacker can get code execution

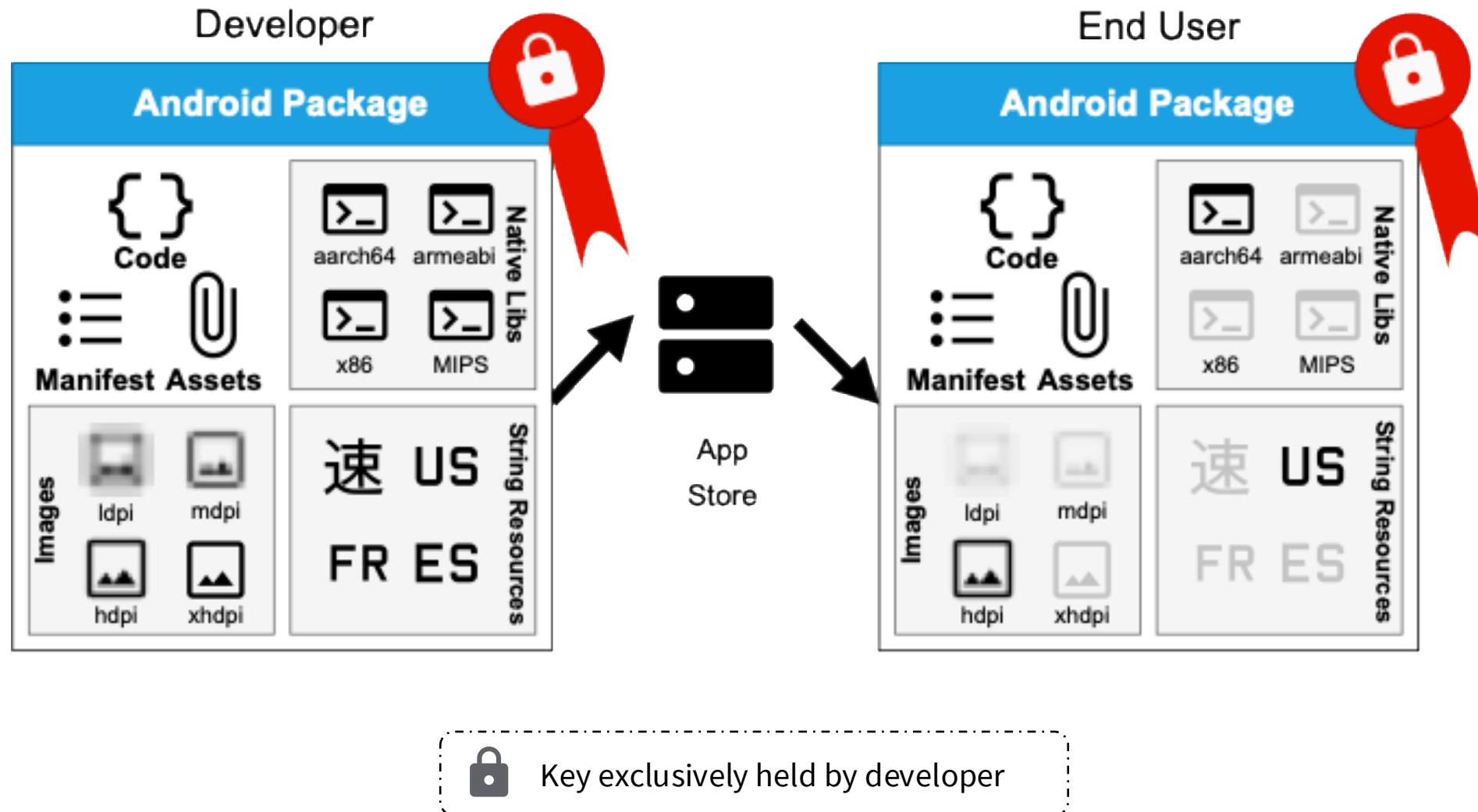
Mobile Supply Chains

Mobile Supply Chains

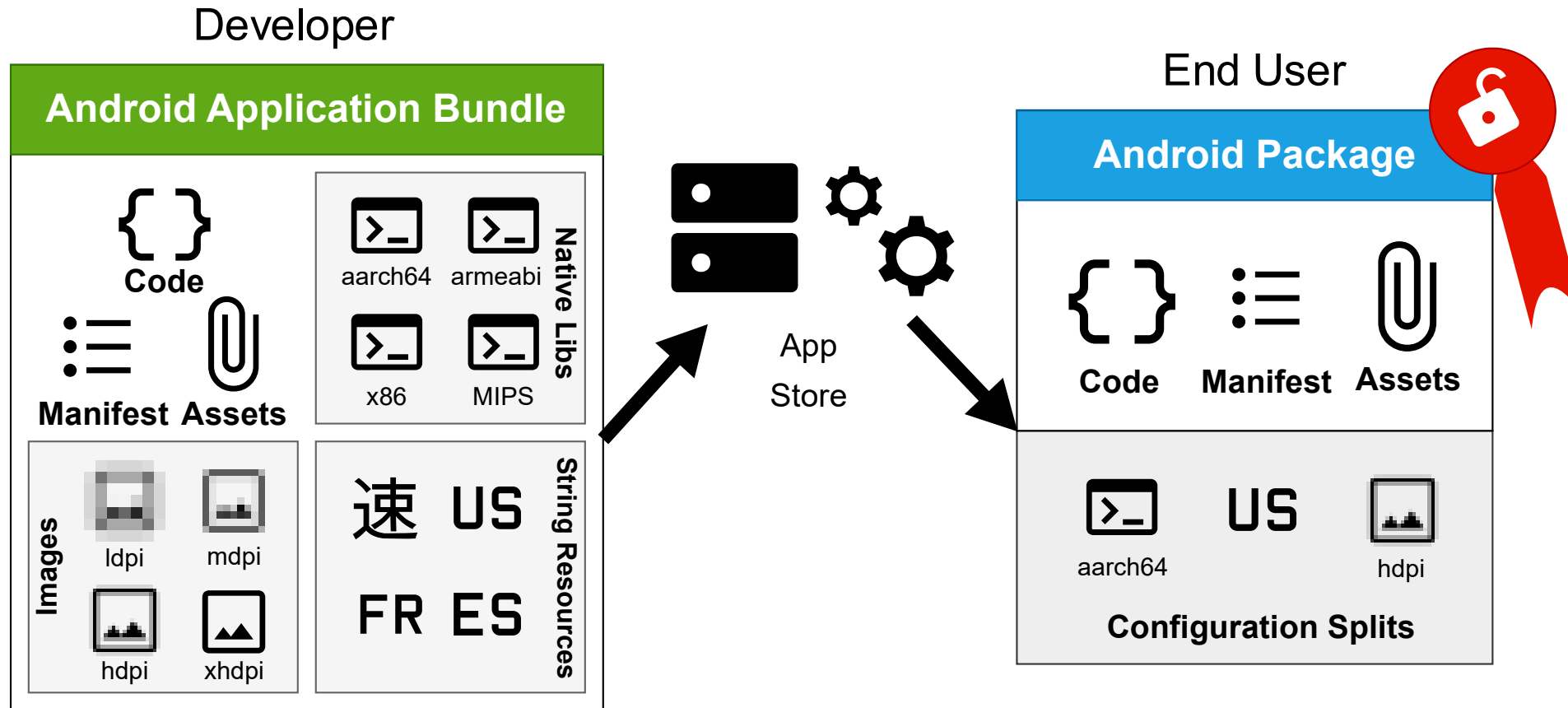
isec.tugraz.at ■



Direct APK Distribution (-2018)



Android App Bundle (AAB; 2018-)

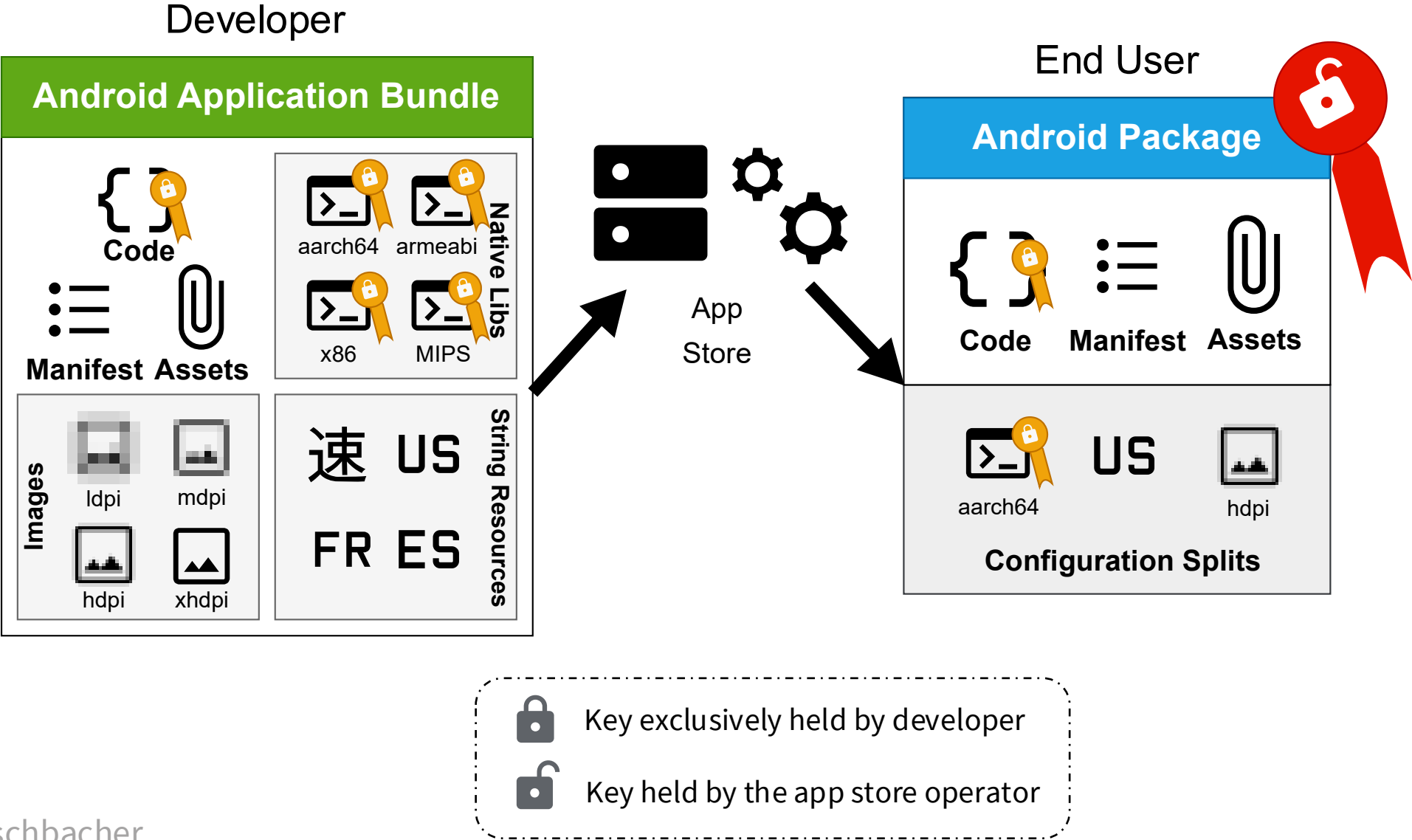


Key exclusively held by developer



Key held by the app store operator

Code Transparency for Android App Bundle (2020-)



We identified design and implementation flaws exploitable in **5 attacks**

Attack	Attacker	Condition
Stripping CT	Any	None
Modifying Assets	Any	Relevant Assets
Inject Shared Library	Privileged / Unprivileged	None
Debuggable Flag	Privileged	None
Backup Opt-In	Privileged	< Android 14

Goal: **Code execution** in context of target app OR **Data extraction**

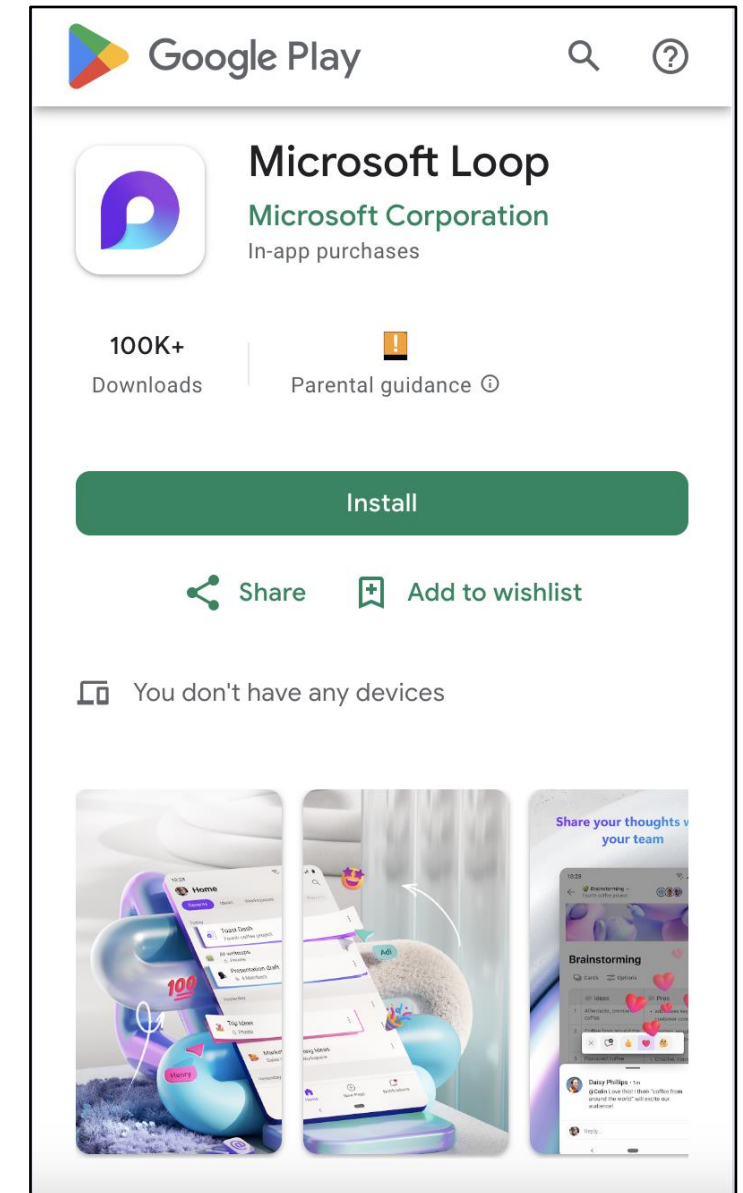
- Apps that use AAB and Code Transparency
 - Google Play: 46% (AAB), 0.0014% (CT) of 3.3m apps
 - Huawei AppGallery: 0.04% (AAB), 0% (CT) of 240k apps
- Executable Assets in Popular Google Play Apps
 - 22% of 6648 apps cannot use CT (DEX/SO in assets)
 - 52% susceptible to code execution through asset manipulation
 - If they used CT

Case Study

- Microsoft Loop (uses Code Transparency)
- Inject shared static library into app manifest:

```
<uses-static-library  
  android:name="com.attack.library"  
  android:version="1"  
  android:certDigest="f7...9d">  
</uses-static-library>
```

- Bundletool still reports intact CT
- Static library executes in context of MS Loop

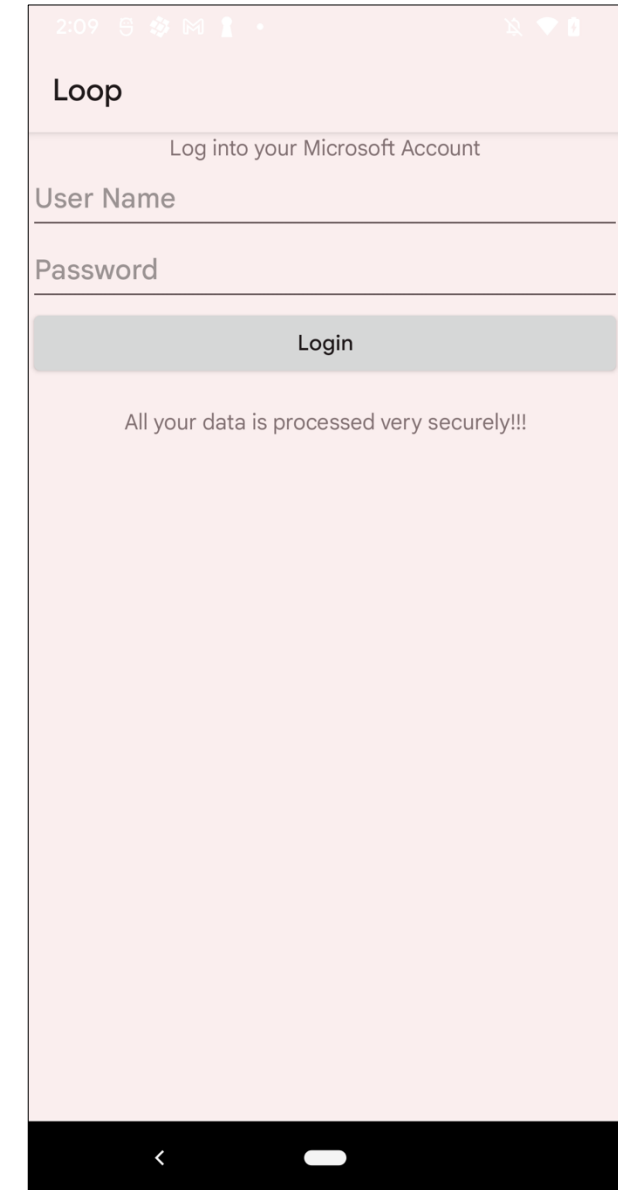


Case Study

- Microsoft Loop (uses Code Transparency)
- Inject shared static library into app manifest:

```
<uses-static-library  
  android:name="com.attack.library"  
  android:version="1"  
  android:certDigest="f7...9d">  
</uses-static-library>
```

- Bundletool still reports intact CT
- Static library executes in context of MS Loop



API Misuse

Vulnerabilities caused by improper integration of (framework) APIs

“The primary cause of these vulnerabilities is the developers’ misunderstanding ...”
- [Georgiev+2012]

The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software

Martin Georgiev
The University of Texas
at Austin

Subodh Iyengar
Stanford University

Suman Jana
The University of Texas
at Austin

Rishita Anubhai
Stanford University

Dan Boneh
Stanford University

Vitaly Shmatikov
The University of Texas
at Austin

ABSTRACT

SSL (Secure Sockets Layer) is the de facto standard for secure Internet communications. Security of SSL connections against an active network attacker depends on correctly validating public-key certificates presented when the connection is established.

We demonstrate that SSL certificate validation is completely broken in many security-critical applications and libraries. Vulnerable software includes Amazon's EC2 Java library and all cloud clients based on it, Amazon's and PayPal's merchant SDKs responsible for transmitting payment details from e-commerce sites to payment gateways, integrated shopping carts such as osCommerce, ZenCart, Ubercart, and PrestaShop, AdMob code used by mobile websites, Chase mobile banking and several other Android apps and libraries, Java Web-services middleware—including Apache Axis, Axis 2, Codehaus XFire, and Pusher library for Android—and all applications employing this middleware. Any SSL connection from any of these programs is insecure against a man-in-the-middle attack.

The root causes of these vulnerabilities are badly designed APIs of SSL implementations (such as JSSSE, OpenSSL, and GnuTLS) and data-transport libraries (such as cURL) which present developers with a confusing array of settings and options. We analyze perils and pitfalls of SSL certificate validation in software based on these APIs and present our recommendations.

Categories and Subject Descriptors

C.2.0 [Computer-Communication Networks]: General—Security and protection; K.4.4 [Computers and Society]: Electronic Commerce—Security

Keywords

SSL, TLS, HTTPS, public-key infrastructure, public-key certificates, security vulnerabilities

1. INTRODUCTION

Originally deployed in Web browsers, SSL (Secure Sockets Layer) has become the de facto standard for secure Internet communi-

cations. The main purpose of SSL is to provide end-to-end security against an active, man-in-the-middle attacker. Even if the network is completely compromised—DNS is poisoned, access points and routers are controlled by the adversary, etc.—SSL is intended to guarantee confidentiality, authenticity, and integrity for communications between the client and the server.

Authenticating the server is a critical part of SSL connection establishment. This authentication takes place during the SSL handshake, when the server presents its public-key certificate. In order for the SSL connection to be secure, the client must carefully verify that the certificate has been issued by a valid certificate authority, has not expired (or been revoked), the name(s) listed in the certificate match(es) the name of the domain that the client is connecting to, and perform several other checks [14, 15].

SSL implementations in Web browsers are constantly evolving through “penetrate-and-patch” testing, and many SSL-related vulnerabilities in browsers have been repaired over the years. SSL, however, is also widely used in non-browser software whenever secure Internet connections are needed. For example, SSL is used for (1) remotely administering cloud-based virtual infrastructure and sending local data to cloud-based storage, (2) transmitting customers’ payment details from e-commerce servers to payment processors such as PayPal and Amazon, (3) logging instant messenger clients into online services, and (4) authenticating servers to mobile applications on Android and iOS.

These programs usually do not implement SSL themselves. Instead, they rely on SSL libraries such as OpenSSL, GnuTLS, JSSSE, CryptoAPI, etc., as well as higher-level data-transport libraries, such as cURL, Apache HttpClient, and arlib, that act as wrappers around SSL libraries. In software based on Web services, there is an additional layer of abstraction introduced by Web-services middleware such as Apache Axis, Axis 2, or Codehaus XFire.

Our contributions. We present an in-depth study of SSL connection authentication in non-browser software, focusing on how diverse applications and libraries on Linux, Windows, Android, and iOS validate SSL server certificates. We use both white- and black-box techniques to discover vulnerabilities in validation logic. Our main conclusion is that SSL certificate validation is completely broken in many critical software applications and libraries. When presented with self-signed and third-party certificates—including a certificate issued by a legitimate authority to a domain called `AtYourSSLAtTheLongToUs`—they establish SSL connections and send their secrets to a man-in-the-middle attacker.

¹SSL also supports client authentication, but we do not analyze it in this paper.

Vulnerabilities caused by improper integration of (framework) APIs

“8% of apps contain some TLS misuse”
- [Fahl+2012]

Why Eve and Mallory Love Android: An Analysis of Android SSL (In)Security

Sascha Fahl, Marian Harbach,
Thomas Muders, Matthew Smith
Distributed Computing & Security Group
Leibniz University of Hannover
Hannover, Germany
{fahl,harbach,muders,smith}@dcsec.uni-
hannover.de

Lars Baumgärtner, Bernd Freisleben
Department of Math. & Computer Science
Philipps University of Marburg
Marburg, Germany
{lbaumgaertner,
freisleb}@informatik.uni-marburg.de

ABSTRACT

Many Android apps have a legitimate need to communicate over the Internet and are then responsible for protecting potentially sensitive data during transit. This paper seeks to better understand the potential security threats posed by benign Android apps that use the SSL/TLS protocols to protect data they transmit. Since the lack of visual security indicators for SSL/TLS usage and the inadequate use of SSL/TLS can be exploited to launch Man-in-the-Middle (MITM) attacks, an analysis of 13,500 popular free apps downloaded from Google's Play Market is presented.

We introduce MalloDroid, a tool to detect potential vulnerability against MITM attacks. Our analysis revealed that 1,074 (8.0%) of the apps examined contain SSL/TLS code that is potentially vulnerable to MITM attacks. Various forms of SSL/TLS misuse were discovered during a further manual audit of 100 selected apps that allowed us to successfully launch MITM attacks against 41 apps and gather a large variety of sensitive data. Furthermore, an online survey was conducted to evaluate users' perceptions of certificate warnings and HTTPS visual security indicators in Android's browser, showing that half of the 754 participating users were not able to correctly judge whether their browser session was protected by SSL/TLS or not. We conclude by considering the implications of these findings and discuss several countermeasures with which these problems could be alleviated.

Categories and Subject Descriptors

C.2.3 [Computer-Communication Network]: Network Operations—Public Networks; H.3.5 [Information Storage and Retrieval]: Online Information Services—Data Sharing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
CCS '12, October 16–18, 2012, Raleigh, North Carolina, USA.
Copyright 2012 ACM 978-1-4503-1651-4/12/10 ...\$10.00.

General Terms

Security, Human Factors

Keywords

Android, Security, Apps, MITMA, SSL

1. INTRODUCTION

Currently, Android is the most used smartphone operating system in the world, with a market share of 48%¹ and over 400,000 applications (apps) available in the Google Play Market², almost doubling the number of apps in only six months³. Android apps have been installed over 10 billion times⁴ and cover a vast range of categories from games and entertainment to financial and business services. Unlike the “walled garden” approach of Apple's App Store, Android software development and the Google Play Market are relatively open and unrestricted. This offers both developers and users more flexibility and freedom, but also creates significant security challenges.

The coarse permission system [9] and over-privileging of applications [16] can lead to exploitable applications. Consequently, several efforts have been made to investigate privilege problems in Android apps [17, 9, 4, 3, 18]. Enck et al. introduced TaintDroid [7] to track privacy-related information flows to discover such (semi-)malicious apps. Bugiel et al. [3] showed that colluding malicious apps can facilitate information leakage. Furthermore, Enck et al. analyzed 1,100 Android apps for malicious activity and detected widespread use of privacy-related information such as IMEI, IMSI, and ICC-ID for “cookie-esque” tracking. However, no other malicious activities were found, in particular no exploitable vulnerabilities that could have led to malicious control of a smartphone were observed [8].

In this paper, instead of focusing on malicious apps, we investigate potential security threats posed by benign Android apps that legitimately process privacy-related user data, such as log-in credentials, personal documents, contacts, financial data, messages, pictures or videos. Many of these apps communicate over the Internet for legitimate reasons and thus request and require the INTERNET permission. It is then

¹<http://android-ssl.org/s/1>

²<http://android-ssl.org/s/2>

³<http://android-ssl.org/s/3>

⁴<http://android-ssl.org/s/4>

Vulnerabilities caused by improper integration of (framework) APIs

“88% of apps that use crypto contain some misuse”
- [Egele+2013]

An Empirical Study of Cryptographic Misuse in Android Applications

Manuel Egele, David Brumley
Carnegie Mellon University
{megele,dbrumley}@cmu.edu

Yanick Fratantonio, Christopher Kruegel
University of California, Santa Barbara
{yanick,chrisc}@cs.ucsb.edu

ABSTRACT

Developers use cryptographic APIs in Android with the intent of securing data such as passwords and personal information on mobile devices. In this paper, we ask whether developers use the cryptographic APIs in a fashion that provides typical cryptographic notions of security, e.g., IND-CPA security. We develop program analysis techniques to automatically check programs on the Google Play marketplace, and find that 10,327 out of 11,748 applications that use cryptographic APIs 88% overall make at least one mistake. These numbers show that applications do not use cryptographic APIs in a fashion that maximizes overall security. We then suggest specific remediations based on our analysis toward improving overall cryptographic security in Android applications.

Categories and Subject Descriptors

D.2.7 Software Engineering: Distribution, Maintenance, and Enhancement – Restructuring, reverse engineering, and reengineering

Keywords

Software Security, Program Analysis

1 Introduction

Developers use cryptographic primitives like block ciphers and message authenticate codes (MACs) to secure data and communications. Cryptographers know there is a right way and a wrong way to use these primitives, where the right way provides strong security guarantees and the wrong way invariably leads to trouble.

In this paper, we ask whether developers know how to use cryptographic APIs in a cryptographically correct fashion. In particular, given code that type-checks and compiles, does the implemented code use cryptographic primitives correctly to achieve typical definitions of security? We assume that developers who use cryptography in their applications make this choice consciously. After all, a developer would not likely try to protect data that they did not believe needed securing.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '13, November 4–8, 2013, Berlin, Germany.
Copyright 2013 ACM 978-1-4503-2477-9/13/11...\$15.00
http://dx.doi.org/10.1145/2508859.2516693.

We focus on two well-known security standards: security against chosen plaintext attacks (IND-CPA) and cracking resistance. For each definition of security, there is a generally accepted right and wrong way to do things. For example, electronic code book (ECB) mode should only be used by cryptographic experts. This is because identical plaintext blocks encrypt to identical ciphertext blocks, thus rendering ECB non-IND-CPA secure. When creating a password hash, a unique salt should be chosen to make password cracking more computationally expensive.

We focus on the Android platform, which is attractive for three reasons. First, Android applications run on smart phones, and smart phones manage a tremendous amount of personal information such as passwords, location, and social network data. Second, Android is closely related to Java, and Java's cryptographic API is stable. For example, the Cipher API, which provides access to various encryption schemes has been unmodified since Java 1.4 was released in 2002. Third, the large number of available Android applications allows us to perform our analysis on a large dataset, thus gaining insight into how developers use cryptographic primitives.

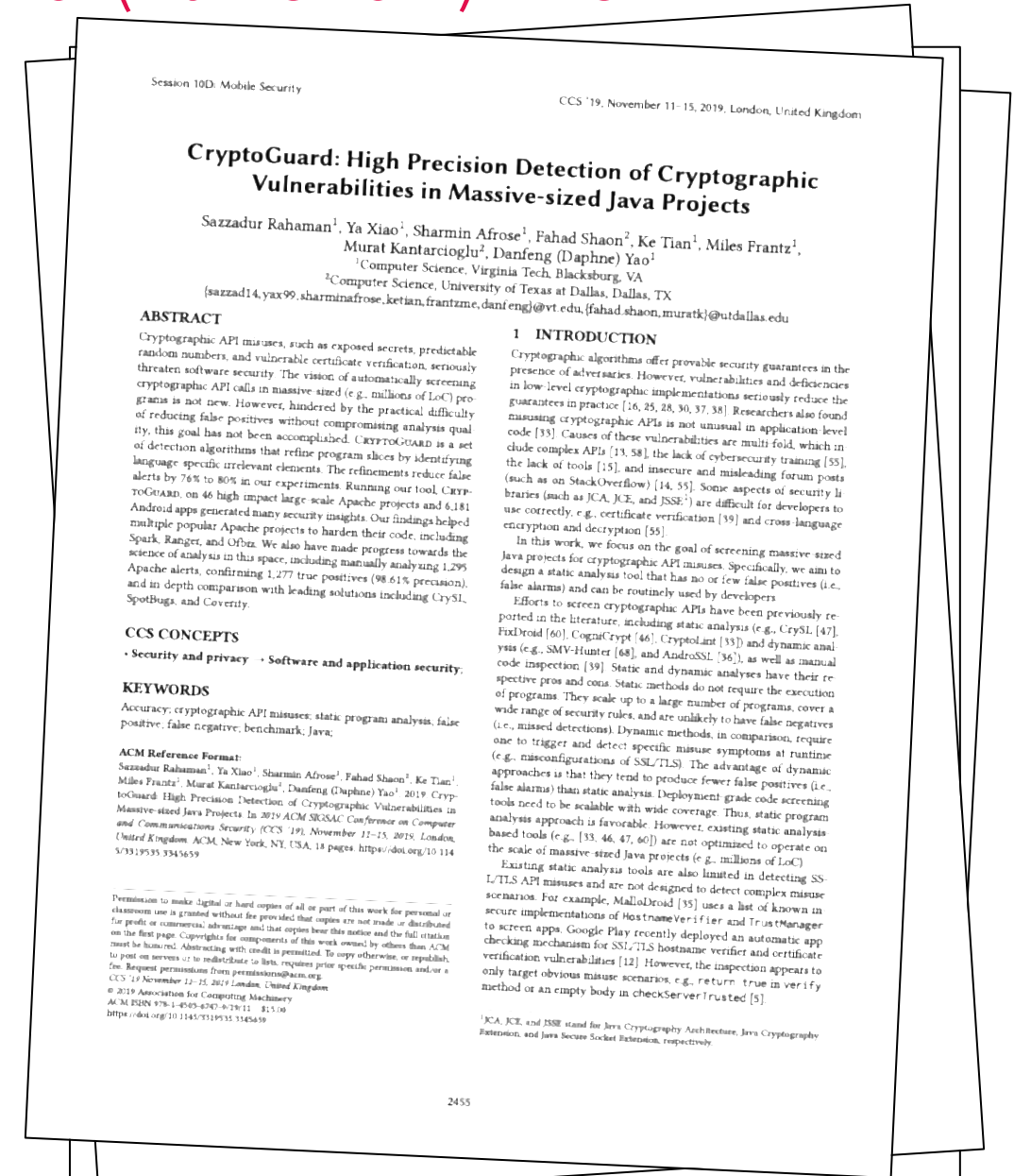
One approach for checking cryptographic implementations would be to adapt verification-based tools like the Microsoft Crypto Verification Kit [7], Murφ [22], and others. The main advantage of verification-based approaches is that they provide strong guarantees. However, they are also heavy-weight, require significant expertise, and require manual effort. The sum of these three limitations make the tools inappropriate for large-scale experiments, or for use by day-to-day developers who are not cryptographers.

Instead, we adopt a light-weight static analysis approach, that checks for common flaws. Our tool, called CRYPTOLINT, is based upon the Androguard Android program analysis framework [12]. The main new idea in CRYPTOLINT is to use static program slicing to identify flows between cryptographic keys, initialization vectors, and similar cryptographic material and the cryptographic operations themselves. CRYPTOLINT takes a raw Android binary, disassembles it, and checks for typical cryptographic misuses quickly and accurately. These characteristics make CRYPTOLINT appropriate for use by developers, app store operators, and security-conscious users.

Using CRYPTOLINT, we performed a study on cryptographic implementations in 11,748 Android applications. Overall we find that 10,327 programs (88% in total) use cryptography inappropriately. The raw scale of misuse indicates a widespread misunderstanding of how to properly use cryptography in Android development.

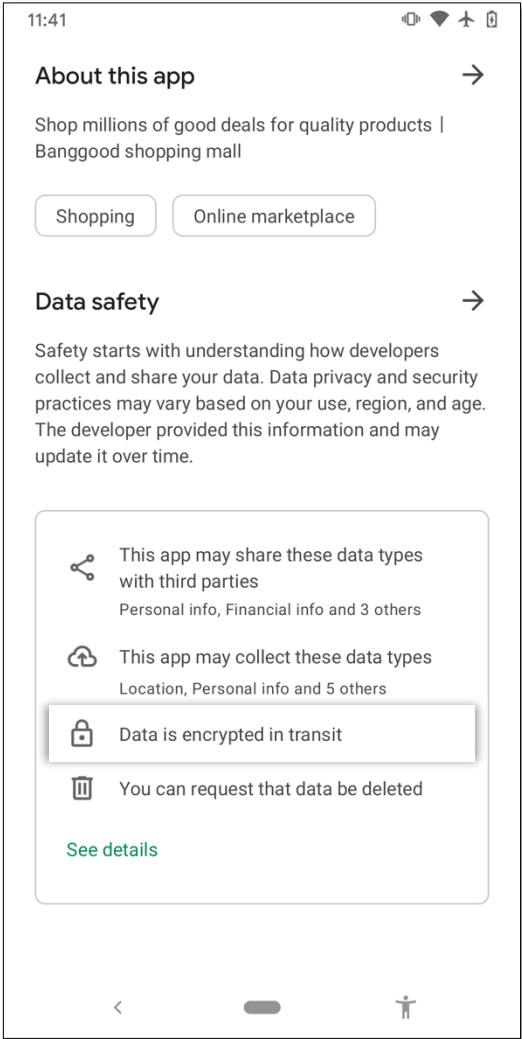
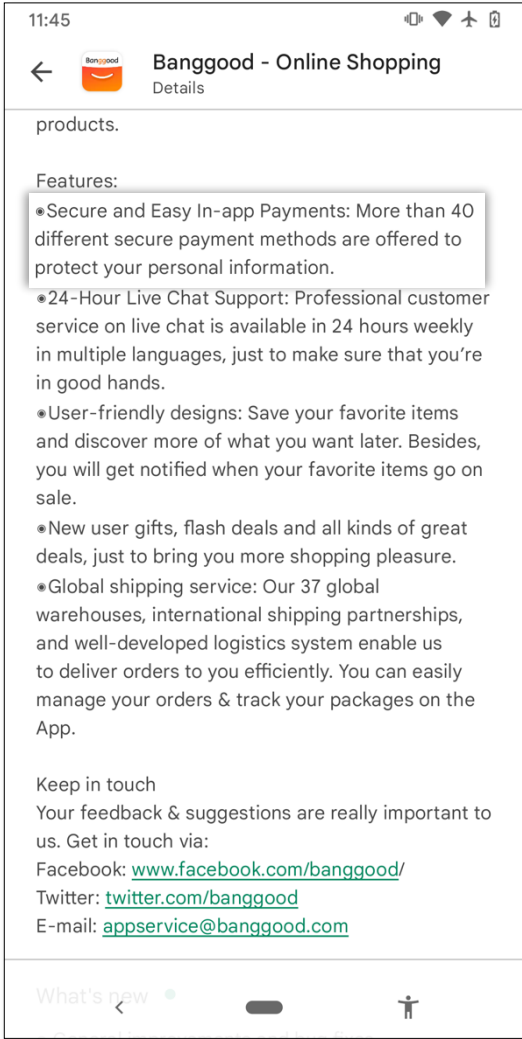
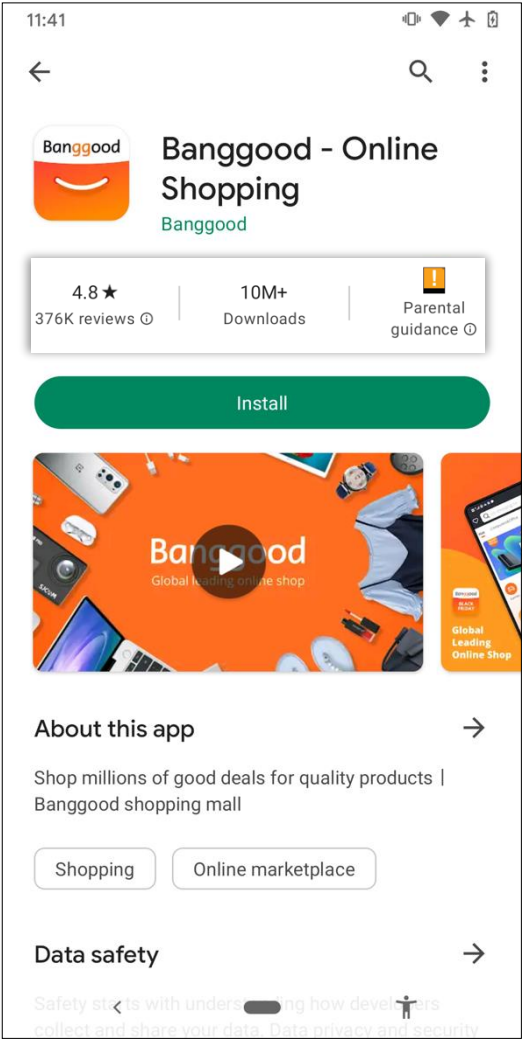
Vulnerabilities caused by improper integration of (framework) APIs

**“91% of top apps
contain some crypto
misuse”
- [Rahman+2019]**



**“99% of top apps
contain some crypto
misuse”**
- [Piccolboni+2021]





Crypto API Misuse

The screenshot shows the Proxyman application interface. At the top, it indicates 'Proxyman | Listening on 192.168.178.150:9090' and 'Free version'. Below the top bar is a filter menu with options: All, HTTP, HTTPS, WebSocket, JSON, Form, XML, JS, CSS, Font, Document, Media, Other. A table of intercepted requests is displayed with columns: ID, URL, Client, Method, Status, Code, Time, Duration, Request, Response, SSL, Edited, and Query Name. The table contains several entries, with the third row (ID 15962) highlighted. Below the table, a detailed view of the selected request is shown. The request is a POST to 'https://m.banggood.com/index.php?com=login&t=register' with a status of '200 OK'. The request body is 'customers_agree=1&email=john.doe%40gmail.com&pwd=SecretPassword&tn_r=77&version=new'. The response is an HTTP/1.1 200 OK with various headers including 'Server: nginx', 'Content-Type: text/html; charset=UTF-8', 'X-Frame-Options: SAMEORIGIN', 'X-XSS-Protection: 1; mode=block', 'P3P: CP="NOI ADM DEV PSAi COM NAV OUR OTRo STP IND DEM"', 'Content-Encoding: gzip', and 'Vary: Accept-Encoding'. At the bottom, there are buttons for 'Clear', 'Filter', and 'Latest', and a status bar showing '1/201 rows selected' and bandwidth usage.

ID	URL	Client	Method	Status	Code	Time	Duration	Request	Response	SSL	Edited	Query Name
15957	https://m.banggood.com/index.php?com=login&t=getUnlockSlider	192.168.178.131	POST	Completed	200	10:56:41.896	254 ms	23 bytes	105,12 KB			
15960	https://m.banggood.com/index.php?com=login&t=checkUnlockStatus	192.168.178.131	POST	Completed	200	10:56:43.937	164 ms	31 bytes	32 bytes			
15962	https://m.banggood.com/index.php?com=login&t=register	192.168.178.131	POST	Completed	200	10:56:44.142	1s 51 ms	83 bytes	515 bytes			
15964	https://m.banggood.com/index.php?com=customer&t=getWishlistId&d...	192.168.178.131	GET	Completed	200	10:56:45.306	182 ms	-	13 bytes			
15965	https://m.banggood.com/index.html?com=index&t=getDynamicData&...	192.168.178.131	GET	Completed	200	10:56:45.326	724 ms	-	18,45 KB			

POST 200 OK https://m.banggood.com/index.php?com=login&t=register

Request Header Query **Body** Form Cookies Raw | Summary Comment + FORM ▾

1 customers_agree=1&email=john.doe%40gmail.com&pwd=SecretPassword&tn_r=77&version=new

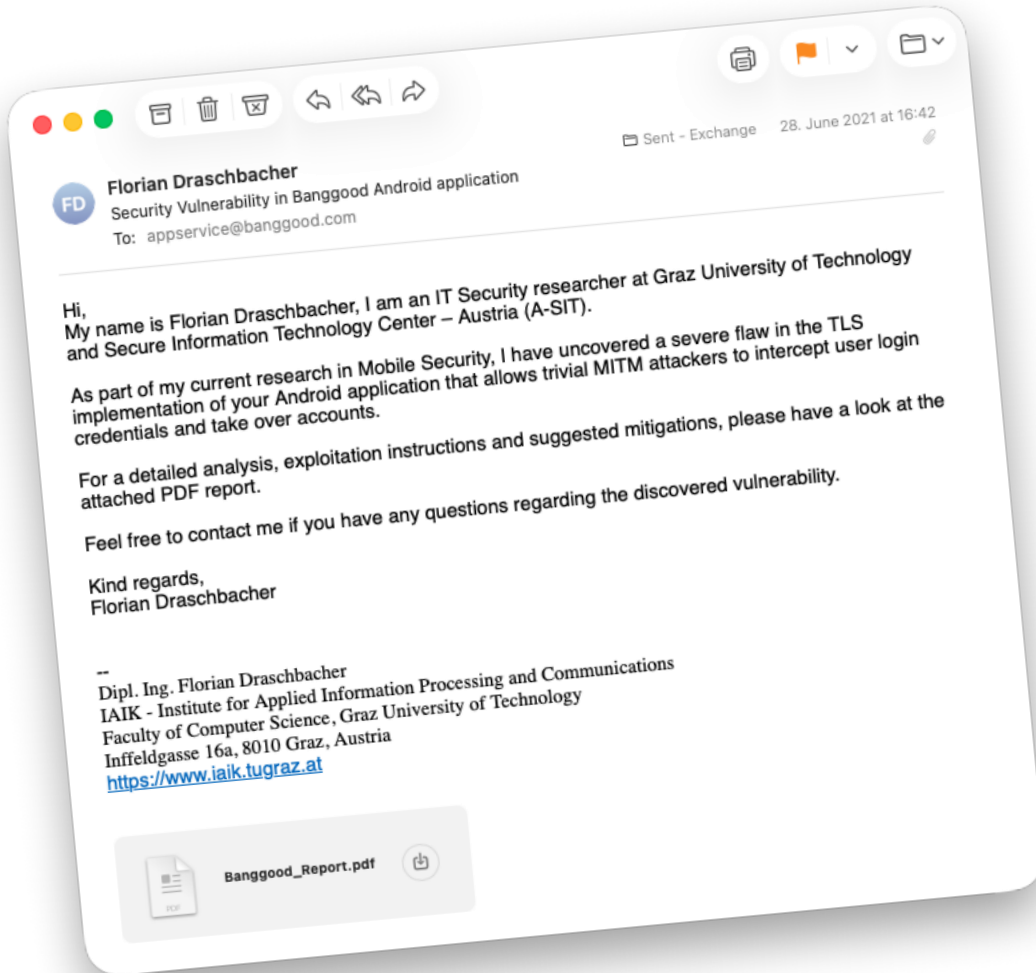
Response Header Body Set-Cookie Raw +

HTTP/1.1 200 OK
Server: nginx
Content-Type: text/html; charset=UTF-8
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
P3P: CP="NOI ADM DEV PSAi COM NAV OUR OTRo STP IND DEM"
Content-Encoding: gzip
Vary: Accept-Encoding

Clear Filter Latest 1/201 rows selected • 269 MB ↑ 2 KB/s ↓ 8 KB/s

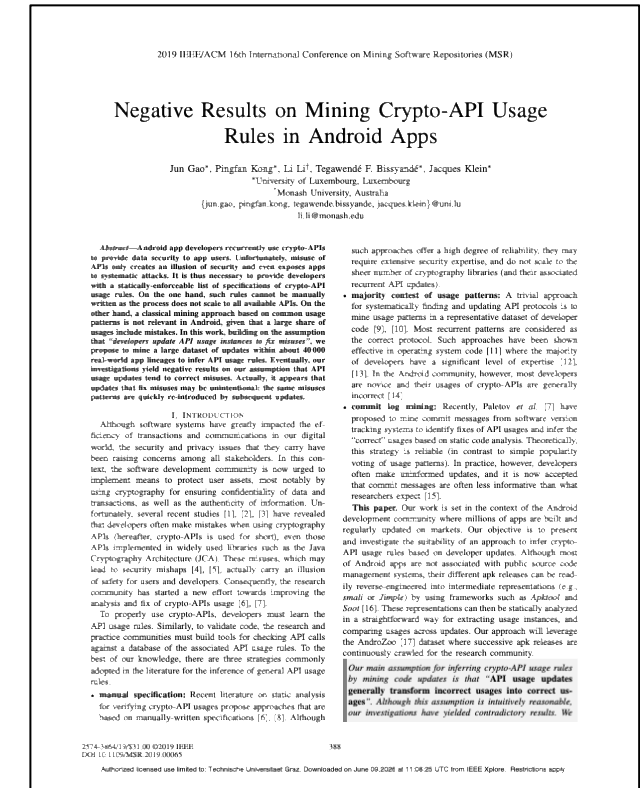
Crypto API Misuse

isec.tugraz.at ■



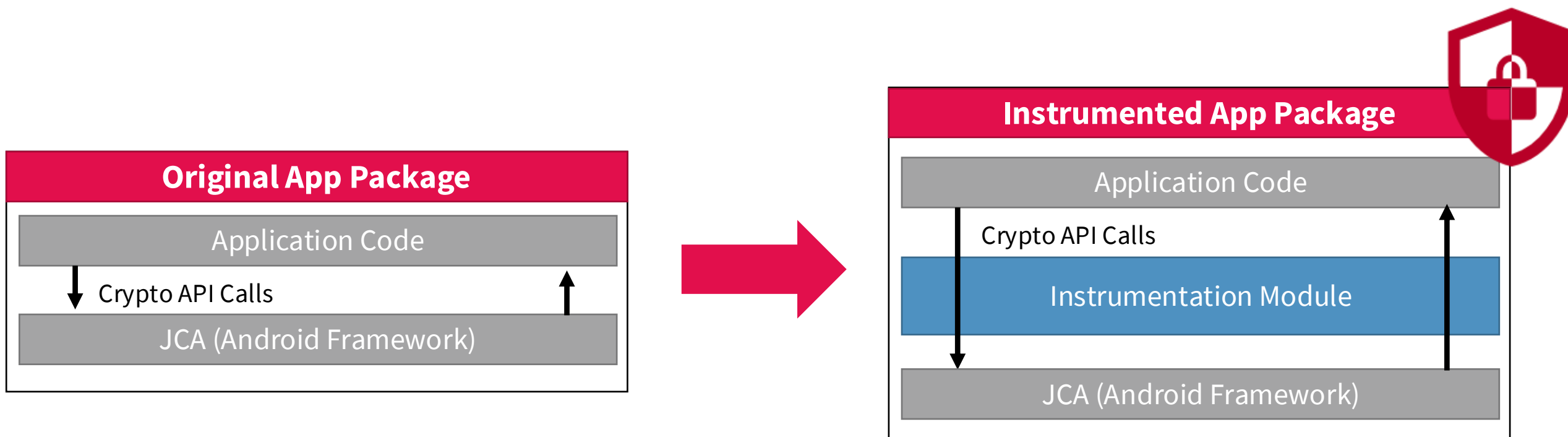
“Developers keep re-introducing (crypto) API misuse into their codebase”
- [Gao+2019]

- Developers unable or unwilling to patch issues
 - High prevalence of API misuse in apps
 - Still no systemic solution for many cases
 - 3rd party mitigations are limited & purpose-specific [Ma+2016]
- ➔ Automated On-Device Mitigation for Crypto API Misuse
- ➔ App-Agnostic Patching Pipeline



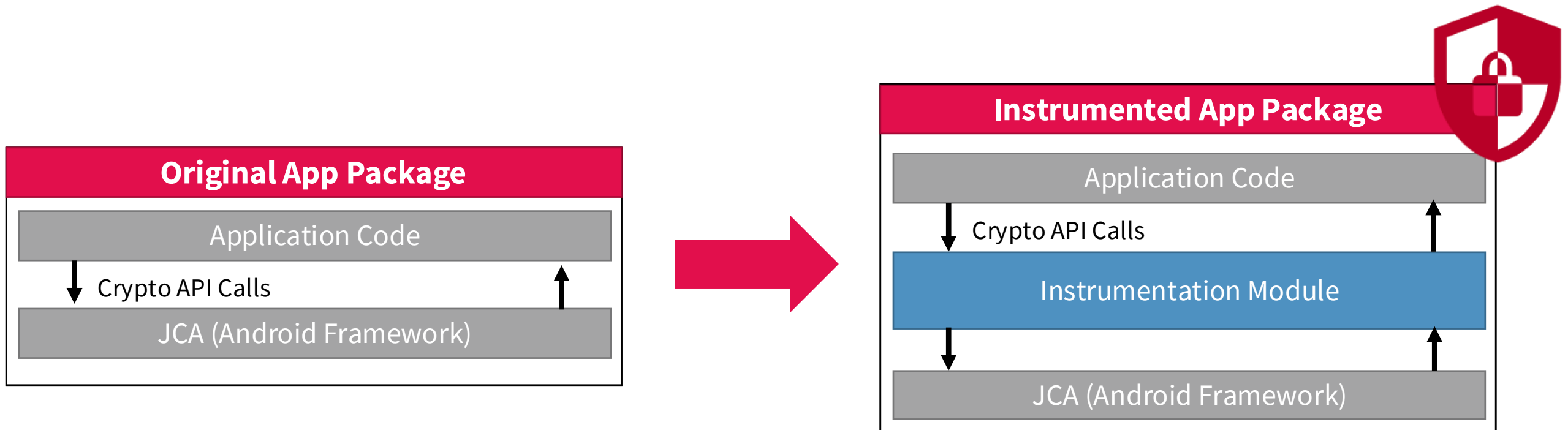
CryptoShield

1. Define mitigation for 10 most critical crypto misuse cases
2. On-device agent that injects instrumentation module



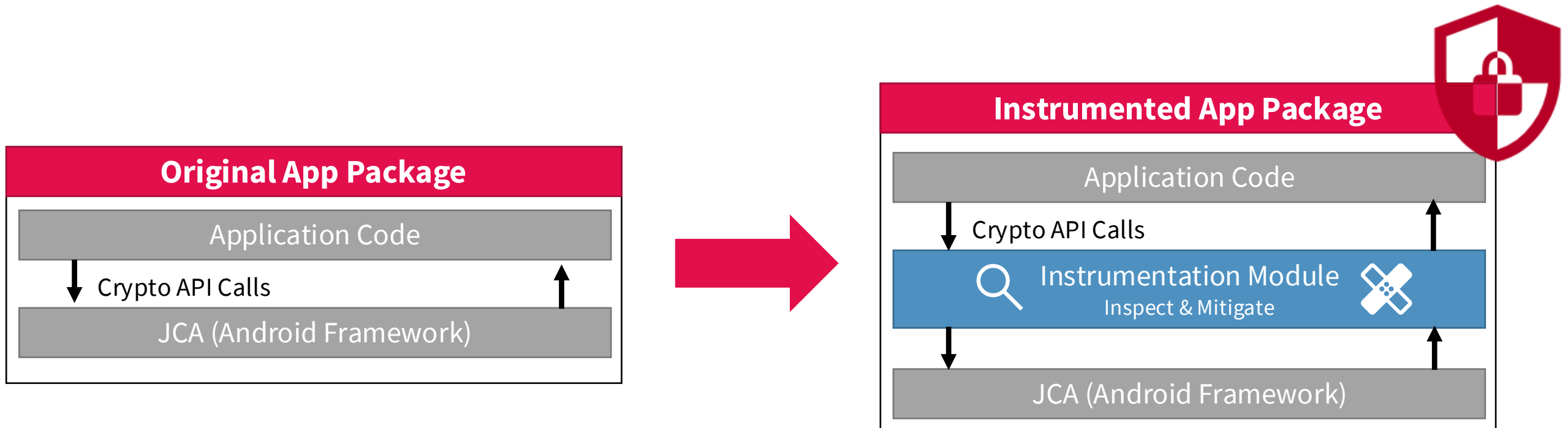
CryptoShield

1. Define mitigation for 10 most critical crypto misuse cases
2. On-device agent that injects instrumentation module



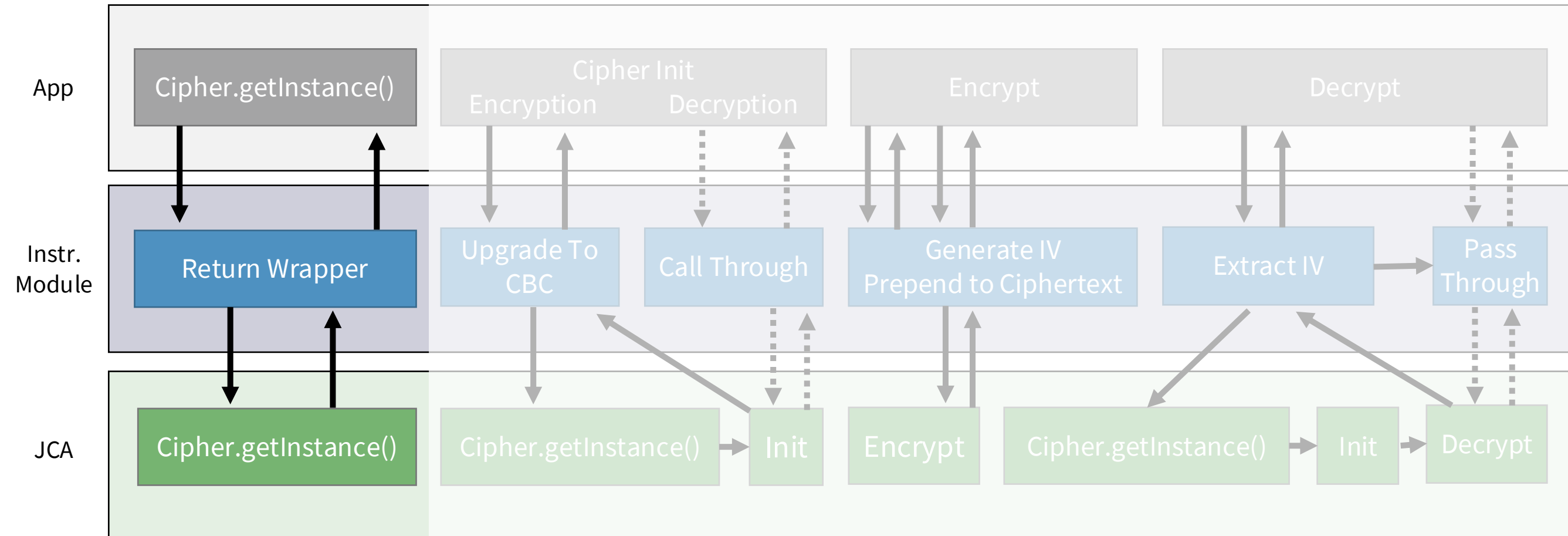
CryptoShield

1. Define mitigation for 10 most critical crypto misuse cases
2. On-device agent that injects instrumentation module



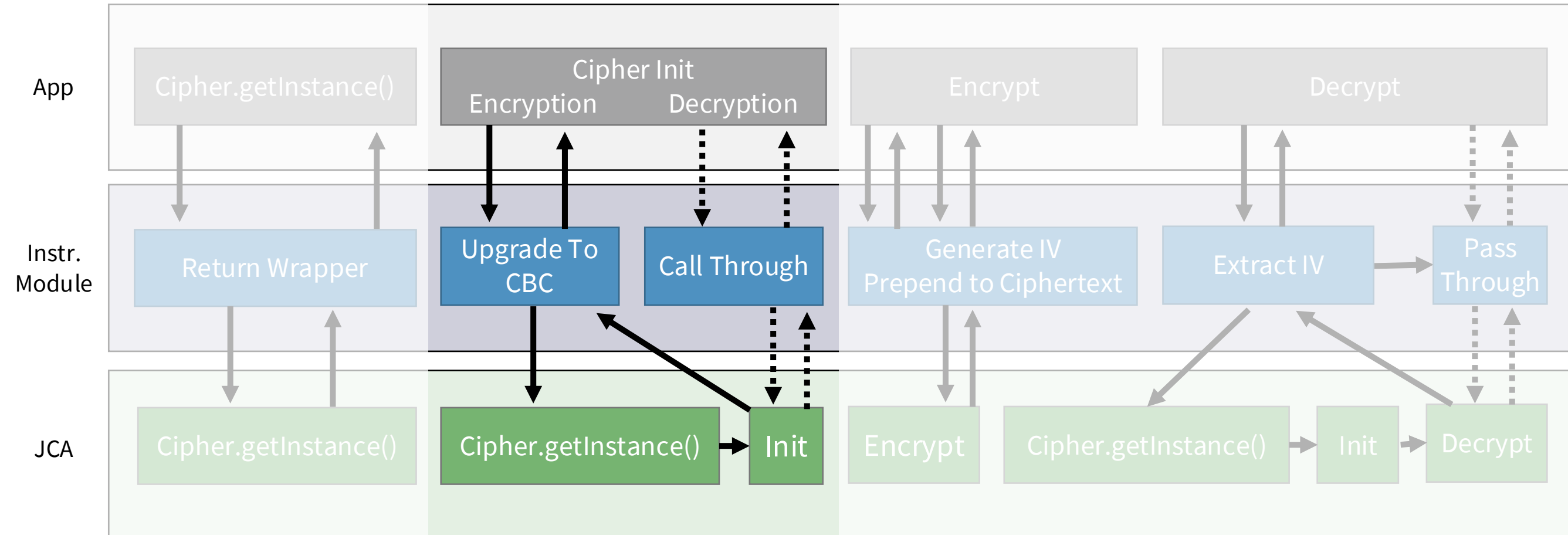
Mitigation Procedures

Misuse Case: Implicit ECB Mode for AES



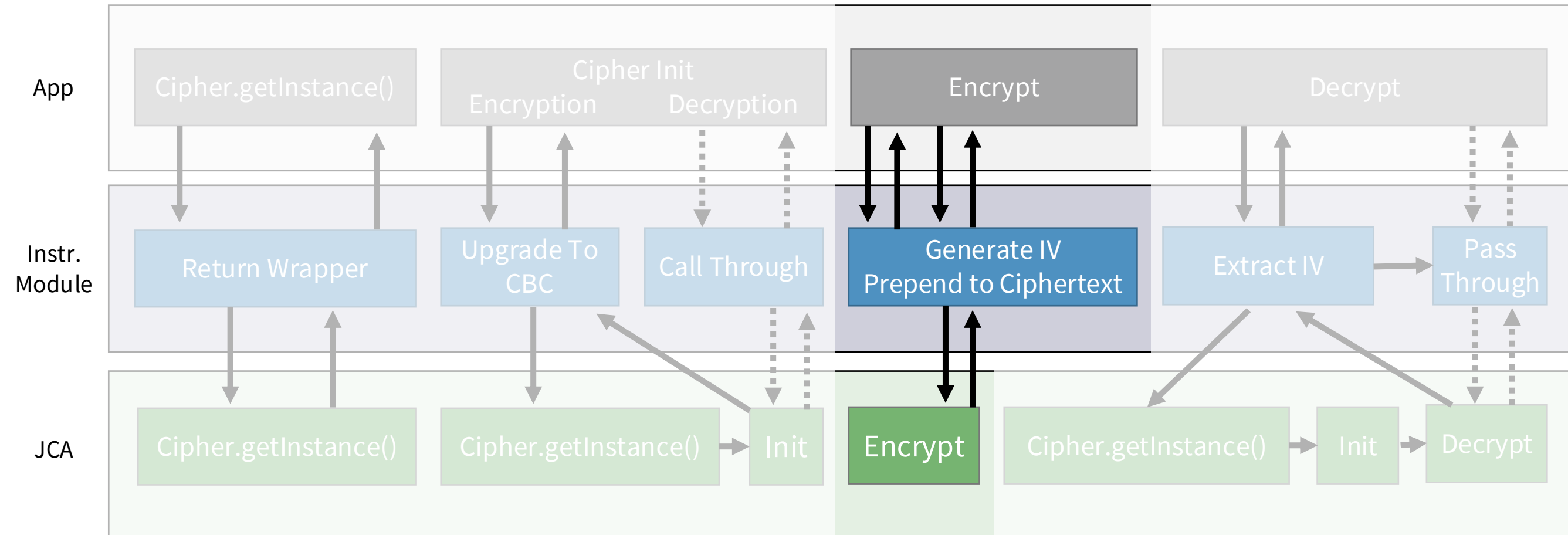
Mitigation Procedures

Misuse Case: Implicit ECB Mode for AES



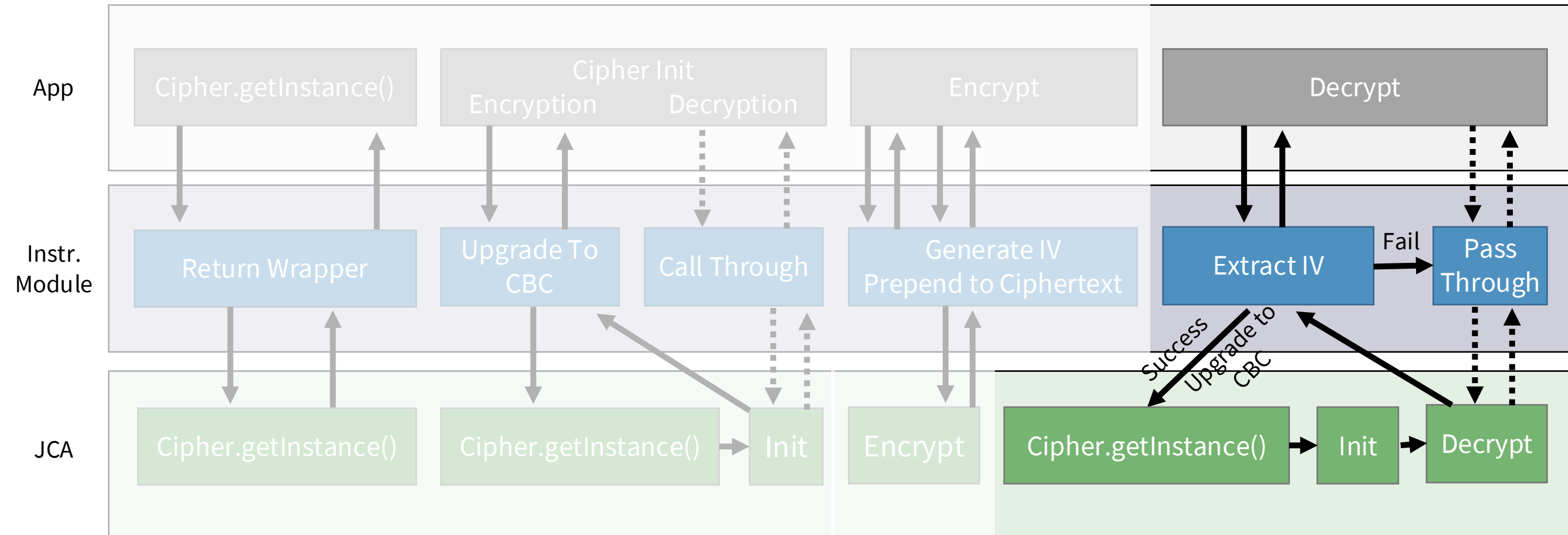
Mitigation Procedures

Misuse Case: Implicit ECB Mode for AES

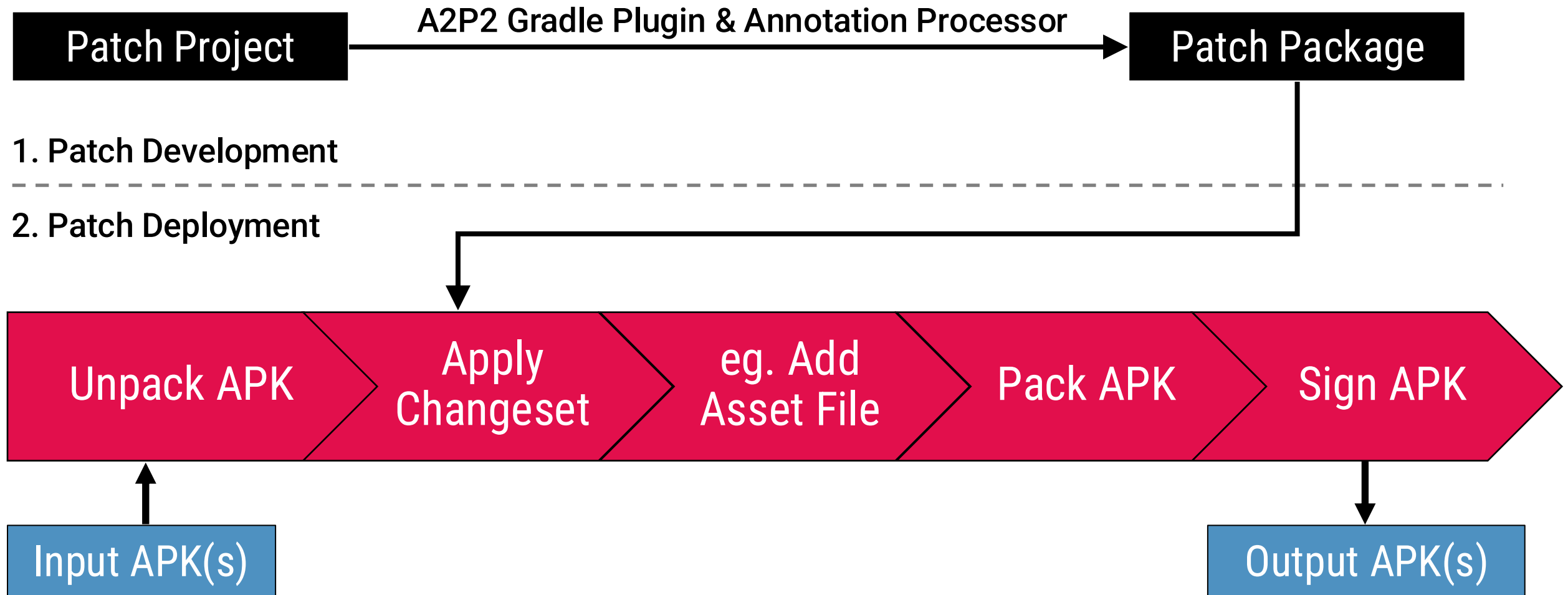


Mitigation Procedures

Misuse Case: Implicit ECB Mode for AES



- Automated analysis
 - Crypto API misuse mitigated in 96 % of vulnerable apps
- Performance
 - Minimal size (5 %) and runtime (100 ms worst-case) overheads
 - Functionality retained for 92 % of 99 popular apps
- Synthetic detection benchmark
 - No FN, better FP than CryptoGuard
- Case studies of 2 critically vulnerable apps



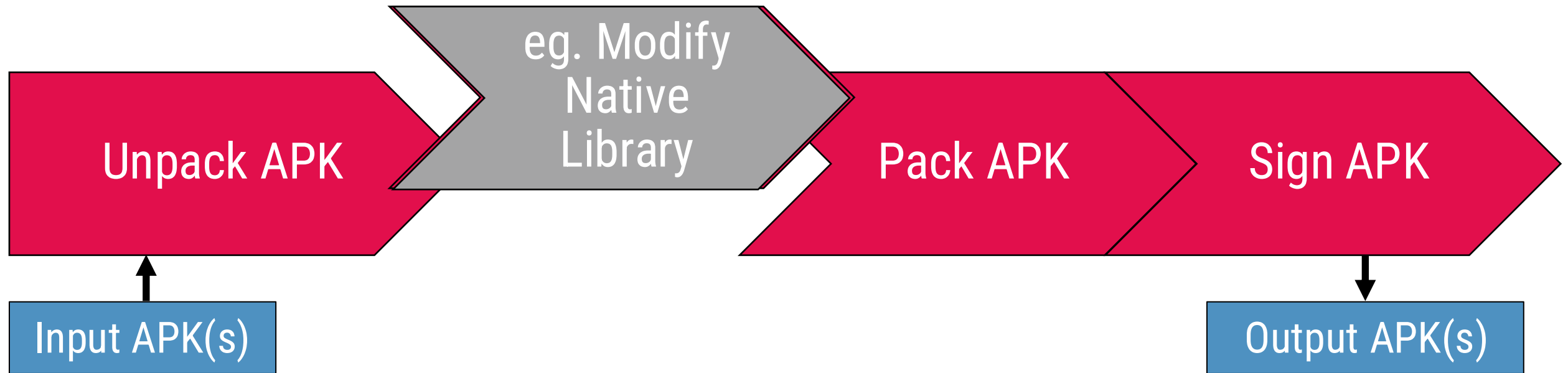
- **Pipeline Architecture** with support for custom stages

```
1 public class TimeTravelPatch {  
2     @PatchClass({"java.lang.System"})  
3     @PatchStaticMethod  
4     public static long currentTimeMillis() {  
5         return OriginalMethods.java_lang_System.currentTimeMillis() - 1000*60*60;  
6     }  
7 }
```

Java Patch

Manifest Patch

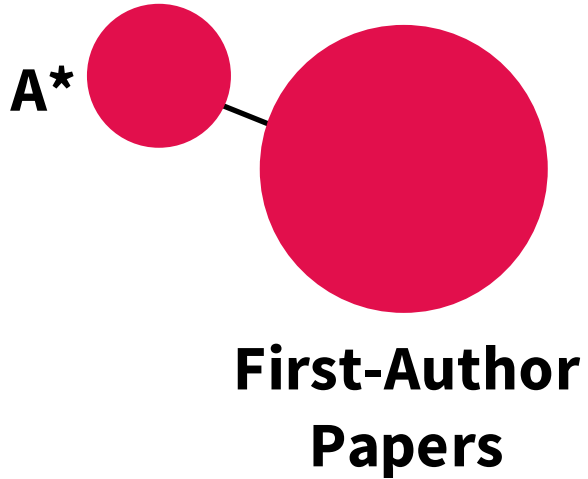
```
1 <?xml version="1.0" encoding="utf-8"?>  
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"  
3     xmlns:patch="http://schemas.android.com/apk/res-auto">  
4  
5     <patch:add sel="manifest">  
6         <meta-data android:name="patched" android:value="true" />  
7     </patch:add>  
8  
9     <patch:replace sel="manifest/@package">${xpath(".").patched}</patch:replace>  
10 </manifest>
```



- **Pipeline Architecture** with support for custom stages

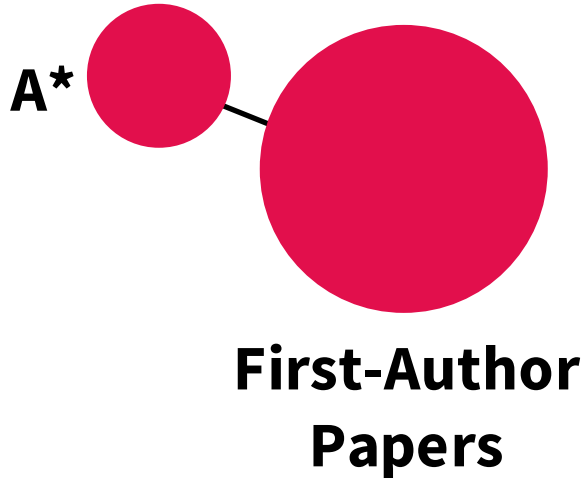
- Minimal App Size Overhead
 - 950 KB + Patch Size
- Negligible Runtime Overhead
 - Per-Call: < 44 ms
 - Launch: < 1.47 ms per method
- High App Compatibility
 - 91 % of 132 most popular Play Store apps
- PoC Patches
 - Securing misuse of dynamic code loading APIs [Falsina+2015], ...

5 Long Years...



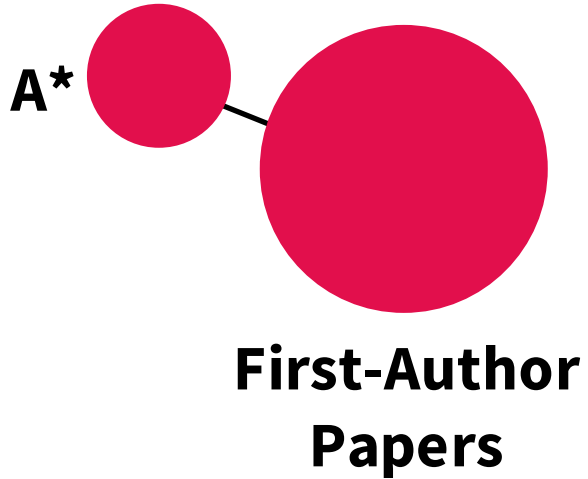
```
Conferences — -zsh — 47x10
[floriandraschbacher@ipn140 Conferences % ls
ACSAC          ESORICS
ARES           EURO S&P
AsiaCCS        NDSS
CCS            PETS
CHES           S&P
DIMVA          USENIX Security
floriandraschbacher@ipn140 Conferences %
```

5 Long Years...



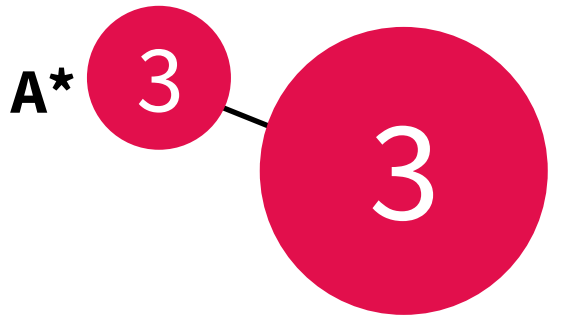
```
Conferences — -zsh — 47x10
[floriandraschbacher@ipn140 Conferences % ls
ACSAC          ESORICS
ARES           EURO S&P
AsiaCCS        NDSS
CCS            PETS
CHES           S&P
DIMVA          USENIX Security
floriandraschbacher@ipn140 Conferences % ls A*
```

5 Long Years...



```
Conferences — -zsh — 47x10
[floriandraschbacher@ipn140 Conferences % ls
ACSAC          ESORICS
ARES           EURO S&P
AsiaCCS        NDSS
CCS            PETS
CHES           S&P
DIMVA          USENIX Security
[floriandraschbacher@ipn140 Conferences % ls A* ]
ACSAC  ARES  AsiaCCS
floriandraschbacher@ipn140 Conferences %
```

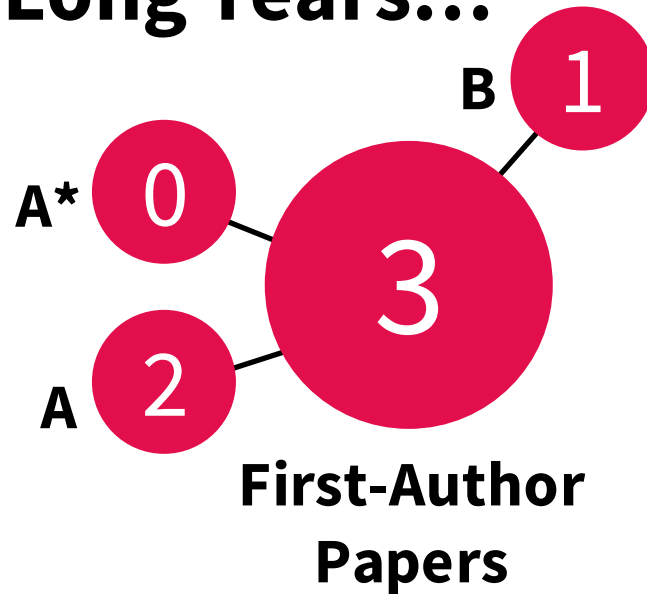
5 Long Years...



**First-Author
Papers**

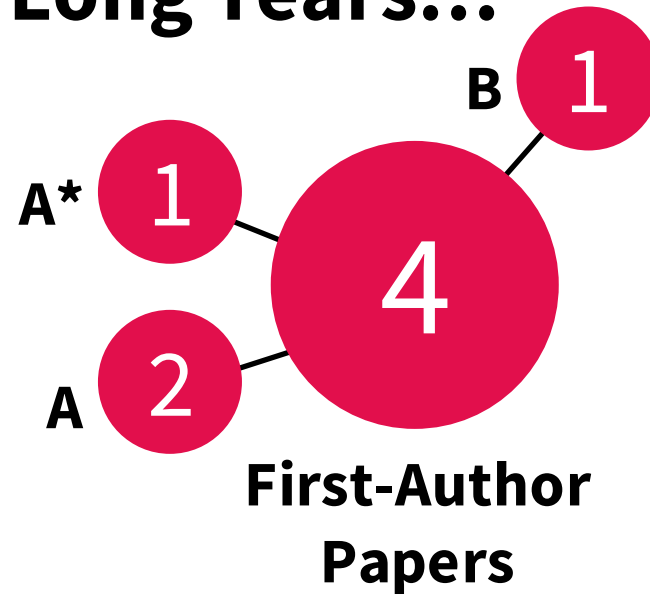
1. **Florian Draschbacher** and Johannes Feichtner. *CryptoShield: Automatic On-Device Mitigation for Crypto API Misuse in Android Applications*. In: AsiaCCS 2023.
2. **Florian Draschbacher**. *A2P2 - An Android Application Patching Pipeline Based On Generic Changesets*. In: ARES 2023.
3. **Florian Draschbacher** and Lukas Maar. *Manifest Problems: Analyzing Code Transparency for Android Application Bundles*. In: ACSAC 2024

5 Long Years...



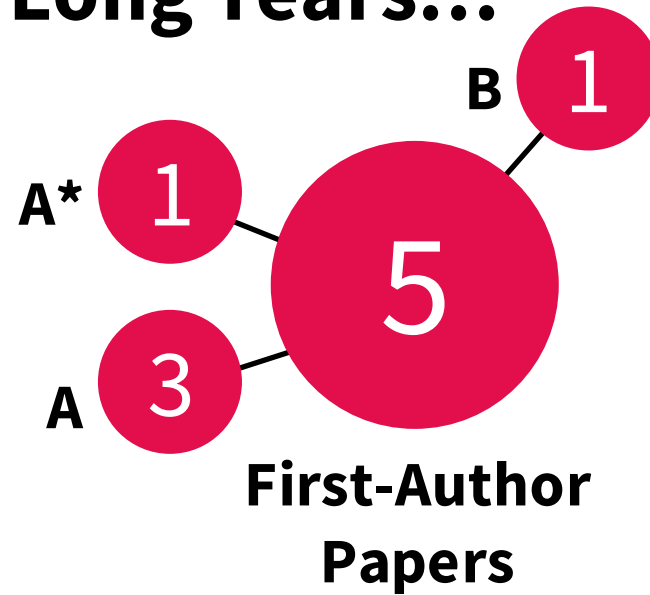
1. **Florian Draschbacher** and Johannes Feichtner. *CryptoShield: Automatic On-Device Mitigation for Crypto API Misuse in Android Applications*. In: AsiaCCS 2023.
2. **Florian Draschbacher**. *A2P2 - An Android Application Patching Pipeline Based On Generic Changesets*. In: ARES 2023.
3. **Florian Draschbacher** and Lukas Maar. *Manifest Problems: Analyzing Code Transparency for Android Application Bundles*. In: ACSAC 2024

5 Long Years...



1. **Florian Draschbacher** and Johannes Feichtner. *CryptoShield: Automatic On-Device Mitigation for Crypto API Misuse in Android Applications*. In: AsiaCCS 2023.
2. **Florian Draschbacher**. *A2P2 - An Android Application Patching Pipeline Based On Generic Changesets*. In: ARES 2023.
3. **Florian Draschbacher** and Lukas Maar. *Manifest Problems: Analyzing Code Transparency for Android Application Bundles*. In: ACSAC 2024
4. **Florian Draschbacher**, Lukas Maar, Mathias Oberhuber, and Stefan Mangard. *ChoiceJacking: Compromising Mobile Devices through Malicious Chargers like a Decade Ago*. In: USENIX Security 2025.

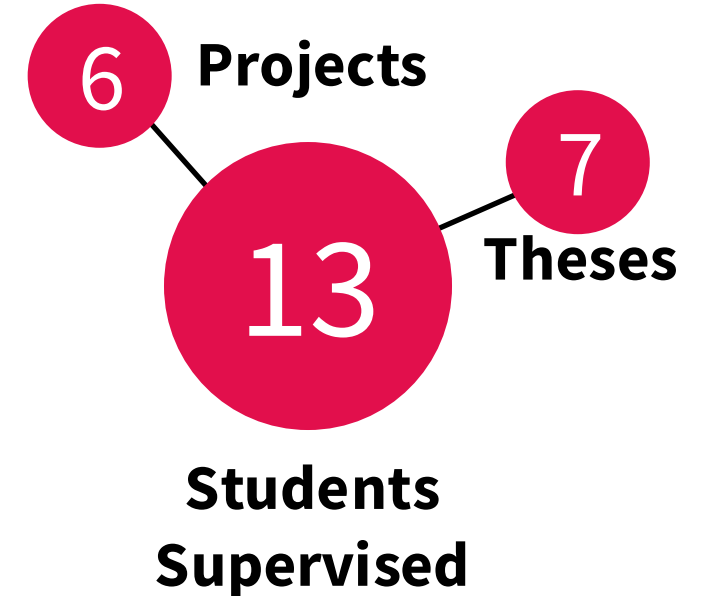
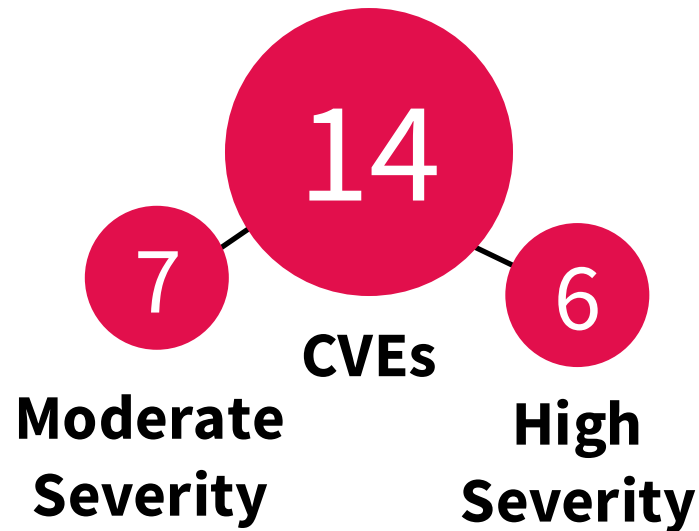
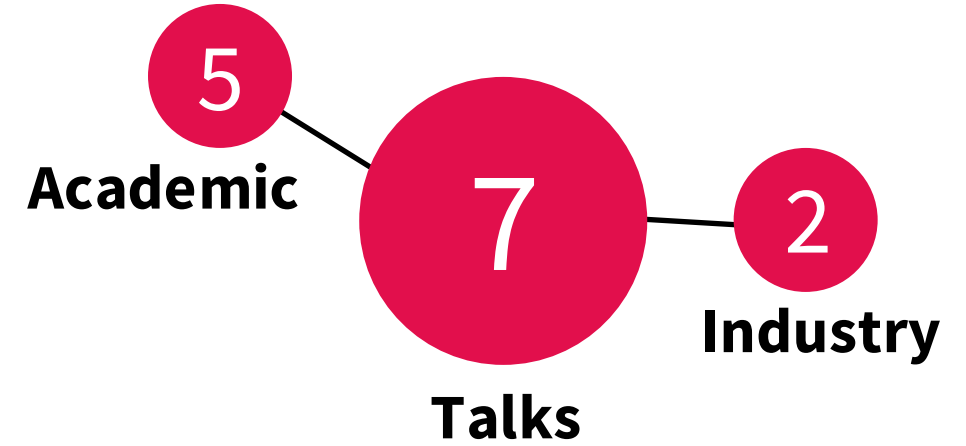
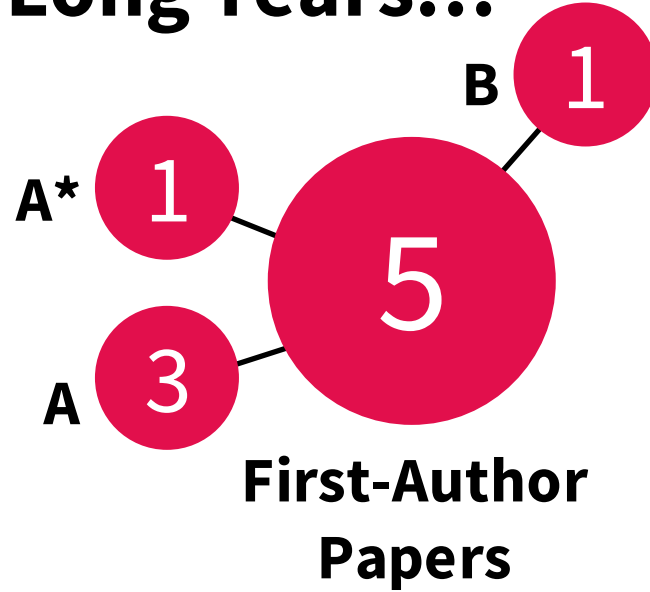
5 Long Years...



1. **Florian Draschbacher** and Johannes Feichtner. *CryptoShield: Automatic On-Device Mitigation for Crypto API Misuse in Android Applications*. In: AsiaCCS 2023.
2. **Florian Draschbacher**. *A2P2 - An Android Application Patching Pipeline Based On Generic Changesets*. In: ARES 2023.
3. **Florian Draschbacher** and Lukas Maar. *Manifest Problems: Analyzing Code Transparency for Android Application Bundles*. In: ACSAC 2024
4. **Florian Draschbacher**, Lukas Maar, Mathias Oberhuber, and Stefan Mangard. *ChoiceJacking: Compromising Mobile Devices through Malicious Chargers like a Decade Ago*. In: USENIX Security 2025.
5. **Florian Draschbacher**, Lukas Maar, Lorenz Schumm, Rene Denifl, Lukas Treffner, and Stefan Mangard. *Clone2Pwn: A Systematic Security Analysis of Data Migration Tools in the Android Ecosystem*. In: ESORICS 2026.

5 Long Years...

isec.tugraz.at



Misconfigured Security Across the Mobile Platform Stack isec.tugraz.at

Apps

API Misuse

❓ CryptoShield

❓ A2P2

Middleware

Supply Chains

❓ Manifest Problems

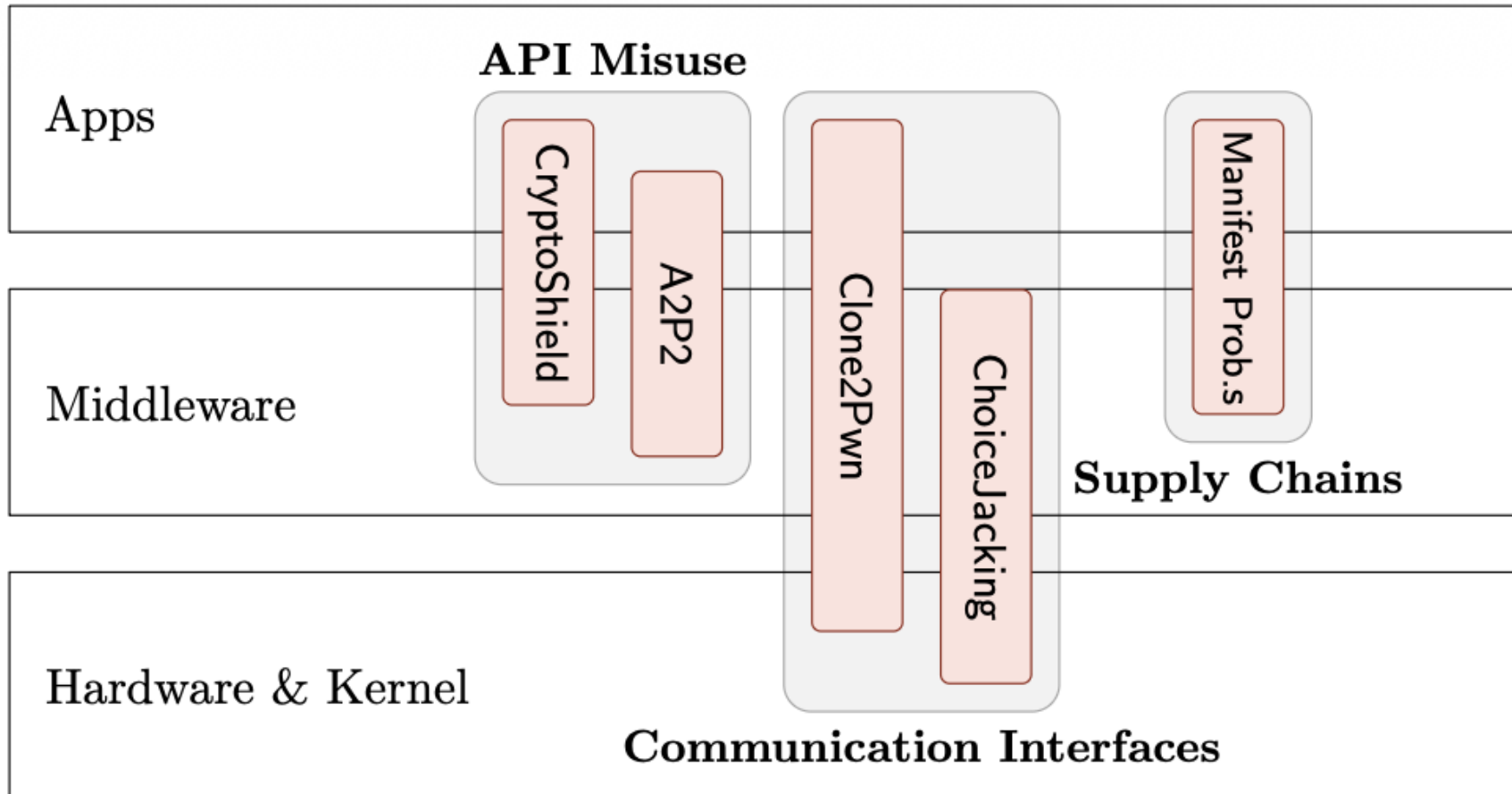
Hardware & Kernel

Communication Interfaces

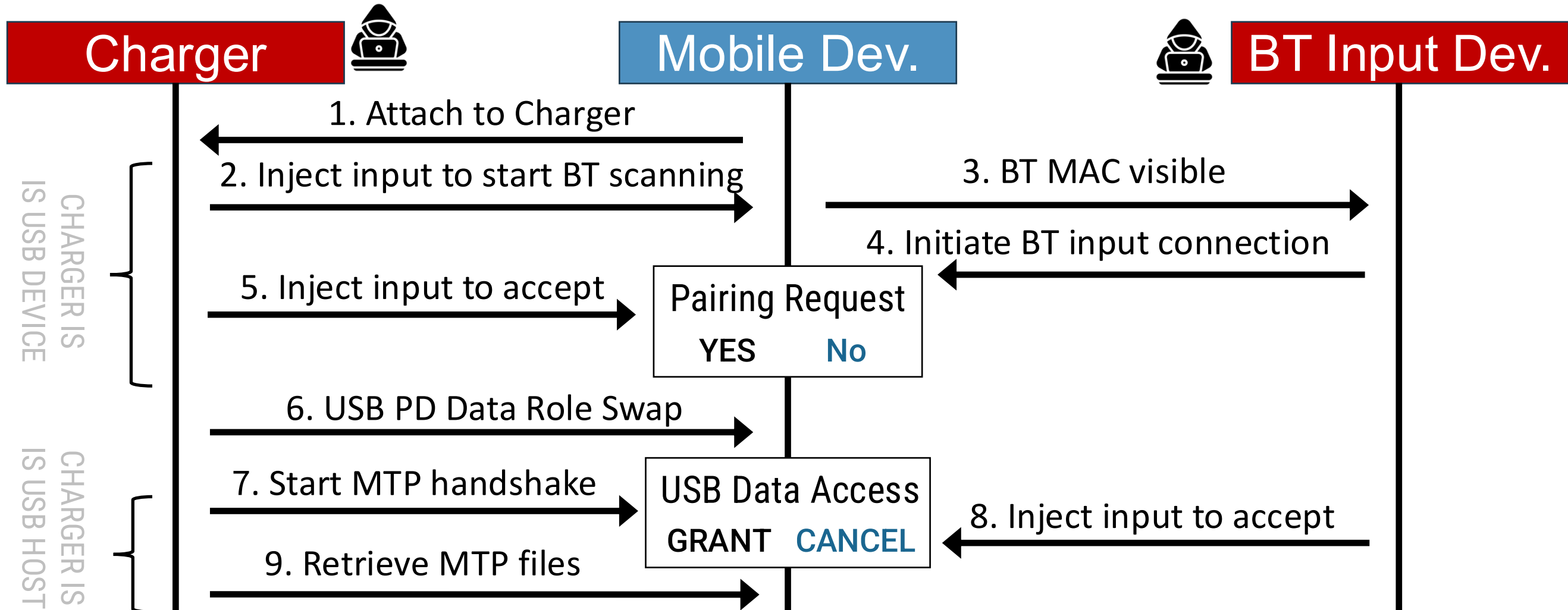
❓ ChoiceJacking

❓ Clone2Pwn

- [Ma+2019] Siqi Ma, Hehao Li, Wenbo Yang, Juanru Li, Surya Nepal, and Elisa Bertino. **Certified Copy? Understanding Security Risks of Wi-Fi Hotspot based Android Data Clone Services.** In: ACSAC 2020
- [Georgiev+2012] Martin Georgiev, Subodh Iyengar, Suman Jana, Rishita Anubhai, Dan Boneh, and Vitaly Shmatikov. **The most dangerous code in the world: validating SSL certificates in non-browser software.** In: CCS 2012
- [Fahl+2012] Sascha Fahl, Marian Harbach, Thomas Muders, Lars Baumgärtner, Bernd Freisleben, and Matthew Smith. **Why eve and mallory love Android: An analysis of Android SSL (in)security.** In: CCS 2012
- [Piccolboni+2021] Luca Piccolboni, Giuseppe Di Guglielmo, Luca P. Carloni, and Simha Sethumadhavan. **CRYLOGGER: Detecting Crypto Misuses Dynamically.** In: S&P 2021
- [Gao+2019] Jun Gao, Pingfan Kong, Li Li, Tegawendé F. Bissyandé, and Jacques Klein. **Negative results on mining Crypto-API usage rules in Android apps.** In: MSR 2019
- [Ma+2016] Siqi Ma, David Lo, Teng Li, and Robert H. Deng. **CDRep: Automatic Repair of Cryptographic Misuses in Android Applications.** In: ASIA CCS 2016
- [Falsina+2015] Luca Falsina, Yanick Fratantonio, Stefano Zanero, Christopher Kruegel, Giovanni Vigna, and Federico Maggi. **Grab 'n Run: Secure and Practical Dynamic Code Loading for Android Applications.** In: ACSAC 2015

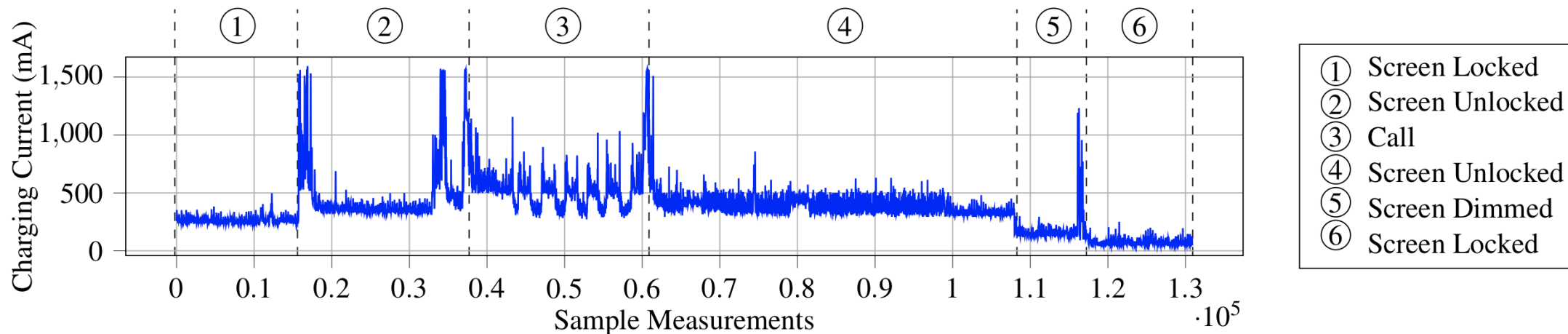


ChoiceJacking: Pivoting via BT HID



ChoiceJacking: Suitable Attack Moments

- Some attacks require unlocked but unattended screen
 - **Suitable moment:** User is in a phone call
- Such moments can be detected from power consumption



- We implemented a PLSC based on one-dimensional CNN classifier

Clone2Pwn: Case Studies

isec.tugraz.at ■



Samsung Smart Switch

Samsung Electronics Co., Ltd.

3.9★

512k reviews ⓘ

1B+

Downloads

- Wi-Fi P2P pairing based on MAC
- App-level AES key sent in plain
- OOB secret only for auth
- Cached files in public storage
- Wi-Fi keys, Samsung token, ...



Android Switch

Google LLC

3.5★

1.2m reviews ⓘ

10B+

Downloads

- WPA2 key [0-9A-F]{8}
- Brute-force in ~30 minutes
- No app-level encryption
- Wi-Fi keys, health records, ...

- **Some Role in Supply Chain**
 - Can manipulate APK delivered to user
- **Unprivileged App Store**
 - Third-party store e.g. installed by user
- **Privileged App Store**
 - Preinstalled with privileged permissions

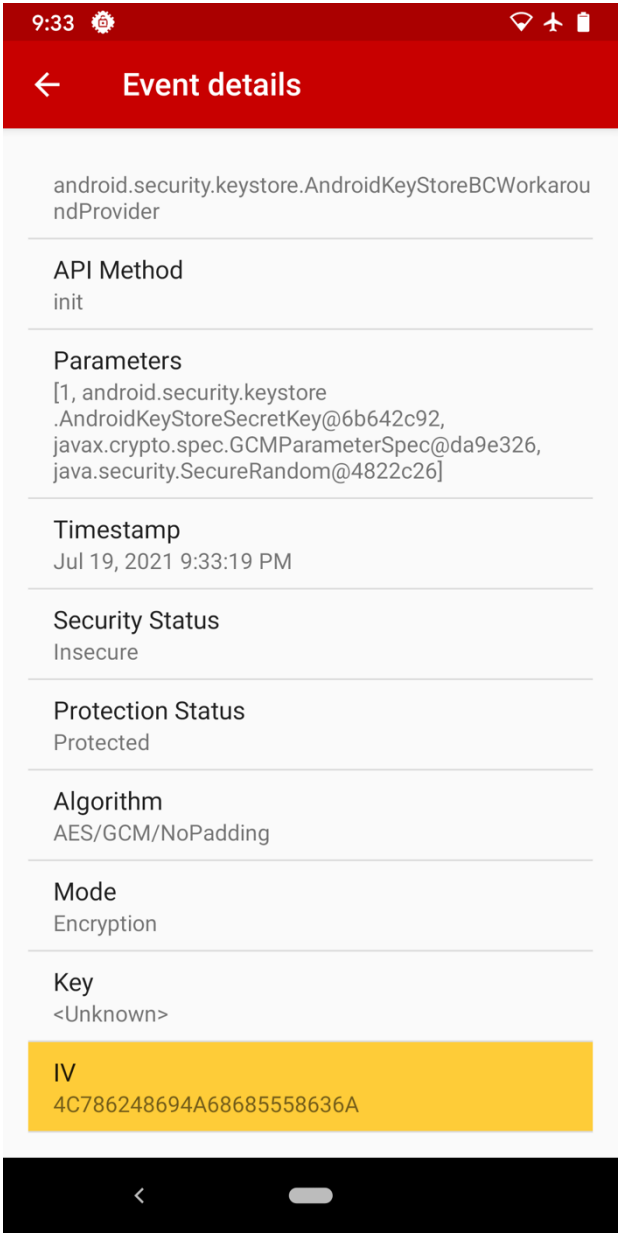
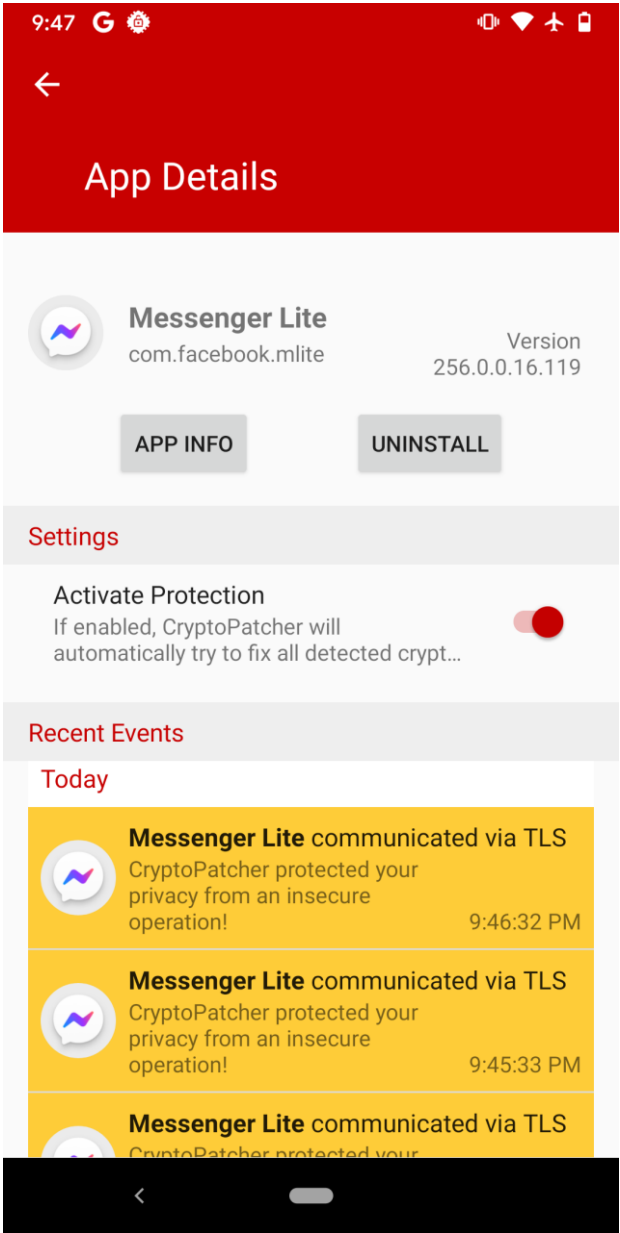
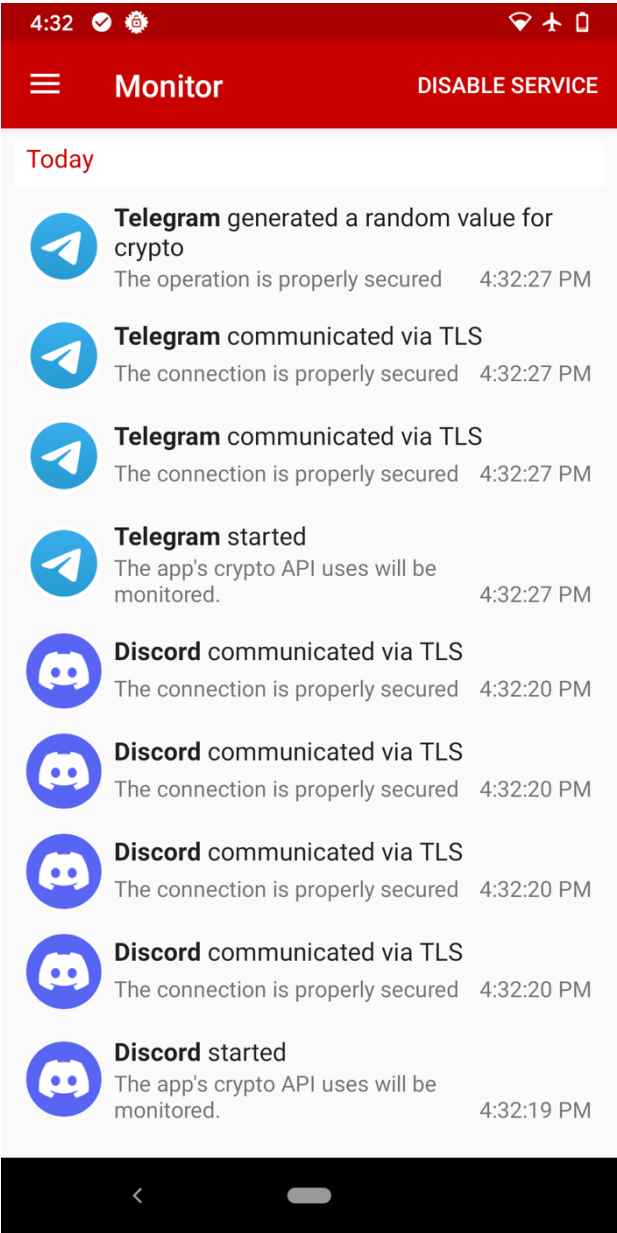
- **R01:** Don't use *unprotected* **HTTP** connections
- **R02:** Don't verify **TLS** connections in *insecure* ways
- **R03:** Don't use *predictable* **passwords** for **key stores**
- **R04:** Don't use *predictable* **passwords** for **key derivations**
- **R05:** Don't use *ECB* mode for **symmetric ciphers**
- **R06:** Don't use *predictable* **IVs** for **symmetric ciphers**
- **R07:** Don't *reuse* **IVs** for **symmetric ciphers**
- **R08:** Don't use *predictable* **salt** values for **key derivations**
- **R09:** Don't *reuse* **salt** values for **key derivations**
- **R10:** Don't use *predictable* **CSRNG** seeds

CryptoShield: Implementation

- Itself an Android application
- Instruments **all apps** installed on device
- Uses **Device Admin API** ^[1]
 - Unrooted operation, MDM
- Original app is only **disabled**, kept on device
 - Updates through Google Play

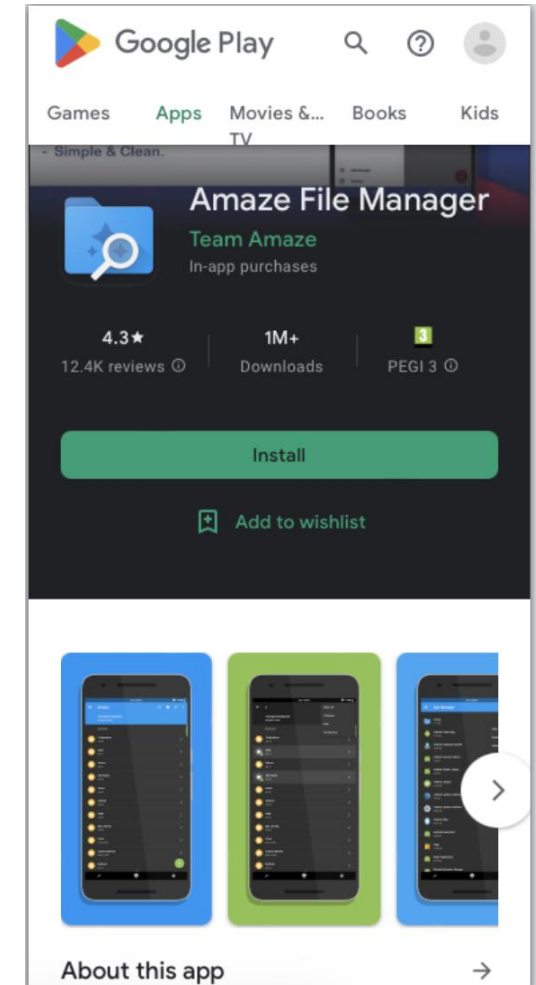
CryptoShield: Agent

isec.tugraz.at



CryptoShield: Bonus Case Study

- Amaze File Manager
 - Open Source, more than 10 million users total
- Used hardcoded IV for all AES-GCM file encryptions
 - $P_2 = C_1 \oplus C_2 \oplus P_1$ (for C_1, C_2 using same key)
 - Any app that guessed any P could decrypt any C
- CryptoShield's mitigation generates fresh IVs
 - Successfully fixes the flaw



A2P2: App Attestation Prevalence

- Analyzed 163,773 top Android apps
- Only **62** of these employed app attestation
- 0.03%

SafetyNOT: On the usage of the SafetyNet Attestation API in Android

Muhammad Ibrahim
Purdue University
USA
ibrahi23@purdue.edu

Abdullah Imran
Purdue University
USA
imran8@purdue.edu

Antonio Bianchi
Purdue University
USA
antonioab@purdue.edu

ABSTRACT

Many apps performing security-sensitive tasks (e.g., online banking) attempt to verify the integrity of the device they are running in and the integrity of their own code. To ease this goal, Android provides an API called the SafetyNet Attestation API, that can be used to detect if the device an app is running in is in a "safe" state (e.g., non-rooted) and if the app's code has not been modified (using, for instance, app repackaging). In this paper, we perform the first large-scale systematic analysis of the usage of the SafetyNet API. Our study identifies many common mistakes that app developers make when attempting to use this API. Specifically, we provide a systematic categorization of the possible misuses of this API, and we analyze how frequent each misuse is. Our results show that, for instance, more than half of the analyzed apps check SafetyNet results *locally* (as opposed to using a remote trusted server), rendering their checks trivially bypassable. Even more surprisingly, we found that none of the analyzed apps invoking the SafetyNet API uses it in a fully correct way.

CCS CONCEPTS

• Security and privacy → Software reverse engineering, Intrusion detection systems.

KEYWORDS

attestation, Android, tampering, SafetyNet, API misuse, reverse engineering

ACM Reference Format:

Muhammad Ibrahim, Abdullah Imran, and Antonio Bianchi. 2021. SafetyNOT: On the usage of the SafetyNet Attestation API in Android. In *The 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21)*, June 24–July 2, 2021, Virtual, WI, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3458864.3466627>

1 INTRODUCTION

Because of Android's high market share, many companies and businesses use Android applications to provide their services but there are characteristics of Android that can be exploited by attackers to compromise Android devices and applications. These characteristics include rooting the device, repackaging applications,



This work is licensed under a Creative Commons Attribution-International 4.0 License.
MobiSys '21, June 24–July 2, 2021, Virtual, WI, USA.
© 2021 Copyright held by the owner/authors.
ACM ISBN 978-1-4505-1440-8/21/07.
<https://doi.org/10.1145/3458864.3466627>

monitoring network traffic, and using unofficial OS distributions. Rooting the device allows users to achieve superuser privileges, change system settings, and gain access to private memory areas. There are tools available for these exploits which make them relatively easier to perform. For this reason, some applications want to check the status of OS and their code. Specifically, they want to check if the OS on the Android device is non-rooted, and the application is not modified. Because by having higher privileges (rooting) and modifying the application, an attacker has much more feasibility to steal information, access premium features, send false information, and compromise the application server.

There are ways of detecting root and repackaging on Android which developers can implement in their applications [19, 46]. However, these methods are shown to be bypassable [28, 37]. Traditionally apps can check for the presence of the binary su (which allows obtaining root privileges) to verify that the device has not been rooted. Complementarily, they can check the contents of the file in which they are stored to verify that they have not been modified. Performing these ad hoc checks has 2 main limitations:

- (1) These checks are complex and error-prone to implement.
- (2) Since these checks are performed in the app's code, an attacker can reverse engineer the app's code and patch it so that it does not perform these checks anymore.

To ease this task and provide a unified, easy-to-use solution, Google implemented a comprehensive API called SafetyNet Attestation API [24] in Android for detecting compromised devices and tampered applications. SafetyNet determines the state of the app and the device integrity in a separate component, running outside the app's process. Apps can call SafetyNet to obtain the results of these checks and send these results to a remote server, where they can check if they are correct. SafetyNet is better than other approaches because on the client side it only requires calling the API and sending the result to the backend server. Developers do not need to implement their own checks that are likely to be incomplete and bypassable. The server can decide what steps to take based on the received information. For example, the server can send an error message and terminate current session with a device if it failed attestation. This helps the authors of an application to implement a system for detecting tampering of their application and the integrity of the devices which are running their application.

However, SafetyNet is not straightforward to use for a developer with no security expertise. SafetyNet must be used in a secure manner and verification of the result must be done on the server side in order to get a correct attestation. If SafetyNet is not used correctly it can be bypassed by attackers. For example, if the verification of the attestation result is done on the application side, an attacker can modify the application to bypass the verification.